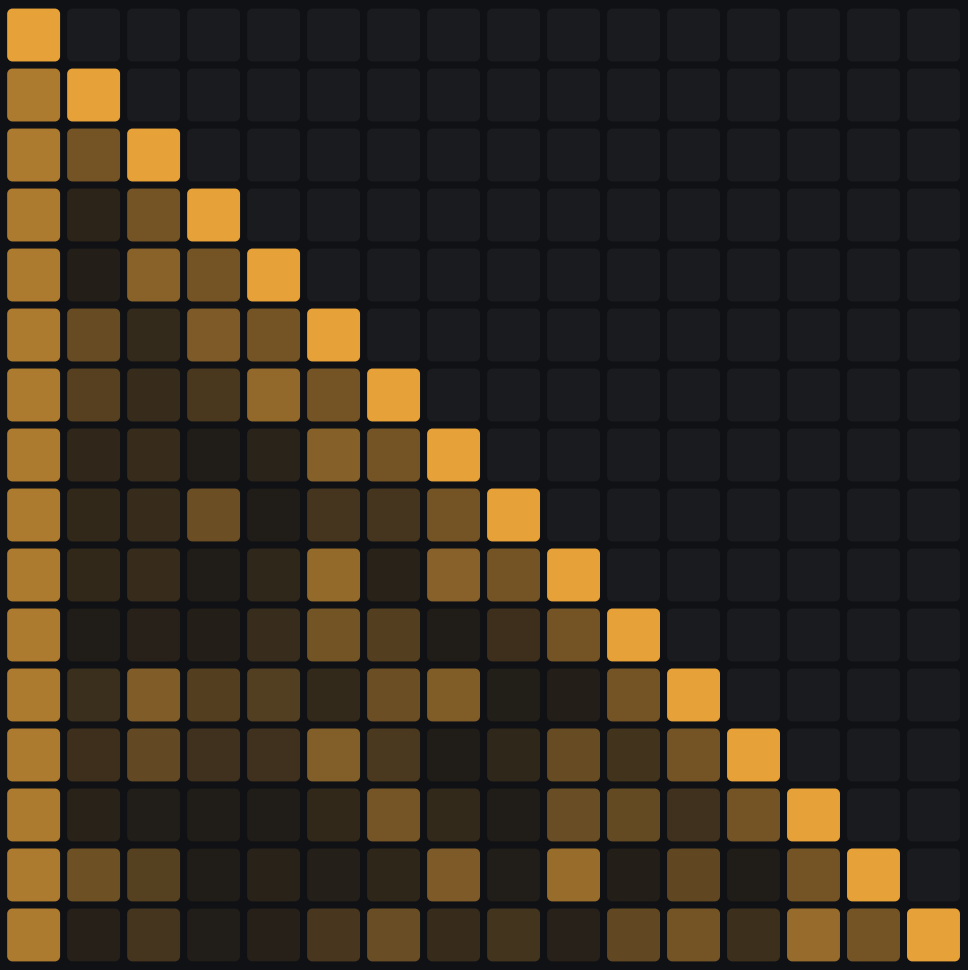
ATTENTION MECHANISMS

EDITION 2026



$$\mathrm{softmax}(QK^{\top} / \sqrt{d} + M) \cdot V$$

注意力机制极简书

从 Bahdanau 到 DeepSeek Sparse Attention
近百种注意力机制的原理、公式与可运行代码

作者：刘玉书

92 节 · 公式 · PyTorch

目录

第 0 章 约定与公共代码 **1**

第 1 章 基础形式 **2**

1.1 加性注意力（Additive / Bahdanau Attention） . . . 2
1.2 乘性注意力（Multiplicative / Luong Attention） . . . 2
1.3 缩放点积注意力（Scaled Dot-Product Attention） . . . 3
1.4 软注意力与硬注意力（Soft vs. Hard） . . . 3
1.5 全局与局部注意力（Global vs. Local） . . . 3
1.6 自注意力与交叉注意力（Self- vs. Cross-Attention） . . . 3
1.7 多头注意力（Multi-Head Attention, MHA） . . . 4
1.8 掩码形式（Masking Patterns） . . . 4

第 2 章 KV 共享与压缩 **6**

2.1 多查询注意力（Multi-Query Attention, MQA） . . . 6
2.2 分组查询注意力（Grouped-Query Attention, GQA） . . . 6
2.3 多头潜注意力（Multi-head Latent Attention, MLA） . . . 6
2.4 张量积注意力（Tensor Product Attention, TPA） . . . 7
2.5 跨层 KV 共享（CLA / YOCO） . . . 7

第 3 章 稀疏注意力 **9**

3.1 固定 / 跨步稀疏（Sparse Transformer） . . . 9
3.2 滑动窗口注意力（Sliding Window Attention, SWA） . . . 9
3.3 Longformer：窗口 + 全局 token . . . 9
3.4 BigBird：窗口 + 全局 + 随机 . . . 10
3.5 膨胀注意力（Dilated Attention, LongNet） . . . 10
3.6 LSH 注意力（Reformer） . . . 10
3.7 路由注意力（Routing Transformer） . . . 11
3.8 注意力汇聚点与 StreamingLLM（Attention Sink） . . . 11
3.9 原生稀疏注意力（Native Sparse Attention, NSA） . . . 11
3.10 块注意力混合（Mixture of Block Attention, MoBA） . . . 12
3.11 DeepSeek 稀疏注意力（DeepSeek Sparse Attention, DSA） . . . 12
3.12 稀疏注意力混合（Mixture of Sparse Attention, MoSA） . . . 13

第 4 章 线性与亚二次注意力 **14**

4.1 线性注意力（Linear Transformer） . . . 14

4.2 Performer（FAVOR+） . . . 15
4.3 Linformer . . . 15
4.4 Nyströmformer . . . 15
4.5 cosFormer . . . 16
4.6 Hedgehog . . . 16
4.7 Based . . . 16
4.8 保持机制（Retention, RetNet） . . . 17
4.9 RWKV（WKV 注意力） . . . 17
4.10 门控线性注意力（Gated Linear Attention, GLA） . . . 18
4.11 DeltaNet 与 Gated DeltaNet . . . 18
4.12 闪电注意力（Lightning Attention） . . . 19
4.13 Kimi Delta Attention（KDA） . . . 19
4.14 状态空间对偶（Mamba-2 / SSD） . . . 19
4.15 对数线性注意力（Log-Linear Attention） . . . 20
4.16 测试时训练层（Test-Time Training, TTT） . . . 20
4.17 Titans：神经长期记忆 . . . 21

第 5 章 对注意力本身的改造 22

5.1 差分注意力（Differential Attention, DIFF Transformer） . . . 22
5.2 门控注意力（Gated Attention） . . . 22
5.3 Sigmoid 注意力 . . . 23
5.4 Softpick . . . 23
5.5 Off-by-One Softmax 与可学习 Sink . . . 23
5.6 遗忘 Transformer（Forgetting Transformer, FoX） . . . 24
5.7 Stick-Breaking 注意力 . . . 24
5.8 多 Token 注意力（Multi-Token Attention, MTA） . . . 24
5.9 Talking-Heads 注意力 . . . 25
5.10 注意力头混合（MoA / MoH） . . . 25
5.11 QK 归一化（QK-Norm） . . . 25
5.12 选择性注意力（Selective Attention） . . . 25
5.13 可扩展 Softmax（Scalable-Softmax, SSMax） . . . 26

第 6 章 位置信息 27

6.1 正弦绝对位置编码（Sinusoidal） . . . 27
6.2 相对位置表示（Shaw 相对位置） . . . 27
6.3 T5 相对位置偏置 . . . 27
6.4 ALiBi（线性偏置注意力） . . . 28
6.5 旋转位置编码（RoPE） . . . 28
6.6 RoPE 长度外推：PI / NTK / YaRN . . . 29
6.7 PaTH 注意力 . . . 29

第 7 章 记忆、检索与潜变量 30

7.1 段级循环（Transformer-XL） . . . 30
7.2 压缩记忆（Compressive Transformer） . . . 30
7.3 kNN 记忆（Memorizing Transformer） . . . 31
7.4 地标注意力（Landmark Attention） . . . 31

7.5 无限注意力（Infini-attention） . . . 31
7.6 潜变量交叉注意力（Perceiver） . . . 32
7.7 诱导点注意力（Set Transformer, ISAB） . . . 32
7.8 槽注意力（Slot Attention） . . . 32

第 8 章 视觉与图结构 **34**
8.1 通道注意力（Squeeze-and-Excitation, SE） . . . 34
8.2 空间注意力与 CBAM . . . 34
8.3 非局部模块（Non-local Neural Networks） . . . 35
8.4 图像块自注意力（ViT） . . . 35
8.5 轴向注意力（Axial Attention） . . . 35
8.6 十字交叉注意力（Criss-Cross Attention, CCNet） . . . 36
8.7 窗口与移位窗口注意力（Swin Transformer） . . . 36
8.8 可变形注意力（Deformable Attention） . . . 36
8.9 图注意力网络（GAT / GATv2） . . . 37
8.10 图 Transformer（Graphormer） . . . 37
8.11 扩散模型中的交叉注意力（Latent Diffusion） . . . 38
8.12 多模态联合注意力（MM-DiT） . . . 38

第 9 章 系统实现层 **39**
9.1 FlashAttention（v1 → v4） . . . 39
9.2 分页注意力（PagedAttention） . . . 39
9.3 环形注意力（Ring Attention） . . . 40
9.4 条纹注意力（Striped Attention） . . . 40
9.5 DeepSpeed-Ulysses . . . 41
9.6 FlexAttention . . . 41
9.7 内存高效注意力（Memory-Efficient Attention / SDPA 后端） . . . 41
9.8 量化注意力（SageAttention） . . . 42

第 10 章 混合架构 **43**
10.1 线性注意力与全注意力交错 . . . 43
10.2 局部与全局注意力交错 . . . 43

附录 速查表 **45**

第 0 章 约定与公共代码

记号：B 批大小，H 头数，N 序列长度，d 每头维度，$D = Hd$ 模型维度。$Q, K, V \in \mathbb{R}^{N\times d}$（省略批与头维）。$M$ 为掩码，可见处为 0，不可见处为 $-\infty$。

一切的母公式：

$$\mathrm{Attn}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}} + M\right)V$$

本书所有机制都可以看成在改这个式子的某一部分：打分函数、归一化函数、掩码 M、K/V 的来源与压缩方式、计算顺序，或者底层实现。

公共代码：后文每个代码块都默认已运行下面这段（`pip install torch` 即可，CPU 上可跑）。

```
import math, torch, torch.nn as nn, torch.nn.functional as F
torch.manual_seed(0)
B, H, N, d = 2, 4, 16, 32                    # 批、头数、序列长度、每头维度
D = H * d                                    # 模型维度
x = torch.randn(B, N, D)                     # 输入序列 (B,N,D)
q, k, v = (torch.randn(B, H, N, d) for _ in range(3))   # 已分好头的 Q/K/V
I, J = torch.arange(N)[:, None], torch.arange(N)[None]  # 查询下标 i、键下标 j
causal = I >= J                              # 因果掩码，True = 可见

def sdpa(q, k, v, mask=None, bias=None):
    """标准缩放点积注意力。mask 中 True 表示可见；bias 加到打分上。"""
    s = q @ k.transpose(-2, -1) / math.sqrt(q.size(-1))
    if bias is not None: s = s + bias
    if mask is not None: s = s.masked_fill(~mask, float('-inf'))
    return s.softmax(-1) @ v

split = lambda t: t.view(t.size(0), t.size(1), H, -1).transpose(1, 2)  # (B,N,D)->(B,H,N,d)
merge = lambda t: t.transpose(1, 2).reshape(t.size(0), t.size(2), -1)  # (B,H,N,d)->(B,N,D)
```

示例代码只为说明原理：用最直白的写法，省略了 dropout、初始化、数值稳定性技巧和高效 kernel。生产环境请使用各论文的官方实现或 `flash-attn`、`fla`（flash-linear-attention）等库。

第 1 章 基础形式

1.1 加性注意力（Additive / Bahdanau Attention）

Bahdanau et al., 2015 · 用于 RNN 机器翻译

最早的神经注意力。用一个小 MLP 给“查询–键”对打分，可以处理 q、k 维度不同的情况。

$$e_{ij} = w^\top \tanh(W_q q_i + W_k k_j), \qquad \alpha_{ij} = \mathrm{softmax}_j(e_{ij}), \qquad o_i = \sum_j \alpha_{ij} v_j$$

```
class AdditiveAttention(nn.Module):
    def __init__(self, dq, dk, h=64):
        super().__init__()
        self.Wq, self.Wk, self.w = nn.Linear(dq, h), nn.Linear(dk, h), nn.Linear(h, 1)
    def forward(self, q, k, v):                       # q:(B,M,dq)  k,v:(B,N,dk)
        e = self.w(torch.tanh(self.Wq(q)[:, :, None] + self.Wk(k)[:, None])).squeeze(-1)
        return e.softmax(-1) @ v                      # (B,M,dv)

out = AdditiveAttention(D, D)(x, x, x); print(out.shape)
```

1.2 乘性注意力（Multiplicative / Luong Attention）

Luong et al., 2015 · dot / general / concat 三种打分

把打分换成内积，计算更快。`general` 在中间加了一个可学习矩阵，`concat` 与加性注意力等价。

$$\mathrm{score}(q,k) = \begin{cases} q^\top k & \text{dot} \\ q^\top W k & \text{general} \\ w^\top \tanh(W[q;k]) & \text{concat} \end{cases}$$

```
W = nn.Linear(D, D, bias=False)
s_dot = x @ x.transpose(1, 2)                         # dot
s_gen = x @ W(x).transpose(1, 2)                      # general
out = s_gen.softmax(-1) @ x; print(out.shape)
```

1.3 缩放点积注意力（Scaled Dot-Product Attention）

Vaswani et al., 2017 · Transformer 的核心

在点积后除以 $\sqrt{d}$，防止维度变大时 softmax 饱和、梯度消失。这是今天几乎所有大模型的默认形式。

$$\mathrm{Attn}(Q,K,V)=\mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

```
out = sdpa(q, k, v)
ref = F.scaled_dot_product_attention(q, k, v)        # PyTorch 内置（会自动选 Flash 等 kernel）
print(torch.allclose(out, ref, atol=1e-5))
```

1.4 软注意力与硬注意力（Soft vs. Hard）

Xu et al., 2015 · Show, Attend and Tell

软注意力对所有位置加权平均，可微。硬注意力只采样一个位置，不可微，需要用 REINFORCE 训练，或用 Gumbel-Softmax 做直通估计。

$$\text{soft: } o=\sum_j \alpha_j v_j \qquad \text{hard: } o=v_{j^*},\ j^*\sim\mathrm{Cat}(\alpha)$$

```
logits = q @ k.transpose(-2, -1) / math.sqrt(d)
soft = logits.softmax(-1) @ v
hard_w = F.gumbel_softmax(logits, tau=1.0, hard=True)  # 前向是 one-hot，反向走软梯度
hard = hard_w @ v
print(hard_w.sum(-1).unique())                          # 每行恰好选中 1 个位置
```

1.5 全局与局部注意力（Global vs. Local）

Luong et al., 2015 · local-p

全局注意力看全部位置。局部注意力先预测一个中心位置 p_t，再用高斯窗口只关注附近的位置。这是后来“滑动窗口注意力”的雏形。

$$p_t=S\cdot\sigma(w_p^\top\tanh(W_p h_t)),\qquad \alpha'_{ts}=\alpha_{ts}\exp\left(-\frac{(s-p_t)^2}{2\sigma^2}\right)$$

```
pos = torch.arange(N).float()
p = N * torch.sigmoid(torch.randn(B, 1, N, 1))          # 每个查询预测的中心（原文由 h_t 计算）
gauss = torch.exp(-(pos - p) ** 2 / (2 * 2.0 ** 2))     # σ = 窗口半宽 / 2
w = (q @ k.transpose(-2, -1) / math.sqrt(d)).softmax(-1) * gauss
out = w @ v; print(out.shape)
```

1.6 自注意力与交叉注意力（Self- vs. Cross-Attention）

Vaswani et al., 2017

自注意力的 Q, K, V 来自同一序列，用来建模序列内部的关系。交叉注意力的 Q 来自一个序列，K, V 来自另一个序列，例如编码器输出、图像或检索到的记忆。

$$\text{Self: Attn}(XW_Q, XW_K, XW_V) \qquad \text{Cross: Attn}(XW_Q,\ YW_K,\ YW_V)$$

```
mha = nn.MultiheadAttention(D, H, batch_first=True)
self_out, _ = mha(x, x, x)                          # 自注意力
mem = torch.randn(B, 10, D)                         # 另一个序列（如编码器输出）
cross_out, _ = mha(x, mem, mem)                     # 交叉注意力
print(self_out.shape, cross_out.shape)
```

1.7 多头注意力（Multi-Head Attention, MHA）

Vaswani et al., 2017

把 D 维切成 H 个子空间，各自独立做注意力，再拼接起来投影回 D 维。不同的头可以学到不同的关系模式。

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \dots, \text{head}_H)W_O, \qquad \text{head}_h = \text{Attn}(XW_Q^h, XW_K^h, XW_V^h)$$

```
class MHA(nn.Module):
    def __init__(self, D, H):
        super().__init__(); self.H = H
        self.qkv, self.o = nn.Linear(D, 3 * D), nn.Linear(D, D)
    def forward(self, x, mask=None):
        q, k, v = self.qkv(x).chunk(3, -1)
        return self.o(merge(sdpa(split(q), split(k), split(v), mask)))

print(MHA(D, H)(x, causal).shape)
```

1.8 掩码形式（Masking Patterns）

因果 / 双向 / Prefix-LM / 块因果

同一个注意力公式，只要换掩码 M，就能得到不同的模型范式：

- 双向：BERT
- 因果：GPT
- Prefix-LM：T5、GLM，前缀部分双向，生成部分因果
- 块因果：块内双向、块间因果，用于扩散语言模型的分块生成、多模态等

$$M_{ij} = \begin{cases} 0 & (i,j) \text{ 可见} \\ -\infty & \text{否则} \end{cases}, \qquad \text{因果: } j \le i; \quad \text{Prefix: } j \le i \ \vee \ j < P$$

```
bidir = torch.ones(N, N, dtype=torch.bool)          # 双向（BERT）
P = 4
prefix = causal | (J < P)                           # Prefix-LM：前 P 个位置所有人可见
bid = torch.arange(N) // 4
```

```
block_causal = bid[:, None] >= bid[None, :]            # 块因果：块内双向、块间因果
for m in (bidir, causal, prefix, block_causal):
    print(sdpa(q, k, v, m).shape, int(m.sum()))
```

第 2 章 KV 共享与压缩

推理时的瓶颈往往不在计算，而在 **KV cache 占用的显存和读取带宽**。本章的方法都在减少每个 token 需要缓存的 K/V 大小。

2.1 多查询注意力（Multi-Query Attention, MQA）

Shazeer, 2019 · PaLM、Falcon 使用

所有查询头共用同一组 K, V，KV cache 缩小为原来的 $1/H$，代价是质量略有下降。

$$\text{head}_h = \text{Attn}(XW_Q^h,\ XW_K,\ XW_V)$$

```
k1, v1 = torch.randn(B, 1, N, d), torch.randn(B, 1, N, d)   # 只有 1 个 KV 头
out = sdpa(q, k1, v1)                                        # 自动广播到 H 个查询头
print(out.shape)
```

2.2 分组查询注意力（Grouped-Query Attention, GQA）

Ainslie et al., 2023 · Llama 2/3、Qwen、Mistral 等主流模型的默认选择

介于 MHA 与 MQA 之间：把 H 个查询头分成 G 组，每组共享一组 KV。$G = H$ 时就是 MHA，$G = 1$ 时就是 MQA。

$$\text{head}_h = \text{Attn}\left(XW_Q^h,\ XW_K^{g(h)},\ XW_V^{g(h)}\right), \qquad g(h) = \lfloor h\,G/H \rfloor$$

```
G = 2
kg, vg = torch.randn(B, G, N, d), torch.randn(B, G, N, d)
out = sdpa(q, kg.repeat_interleave(H // G, 1), vg.repeat_interleave(H // G, 1))
ref = F.scaled_dot_product_attention(q, kg, vg, enable_gqa=True)   # PyTorch ⩾2.5 原生支持
print(torch.allclose(out, ref, atol=1e-5))
```

2.3 多头潜注意力（Multi-head Latent Attention, MLA）

DeepSeek-V2, 2024 · DeepSeek-V3/R1、Kimi K2 使用

把 K、V 联合压缩成一个低维潜向量 c^{KV}，cache 只需要存这个潜向量。推理时可以把 W^{UK} 吸收进 W_Q，免去显式还原 K 的开销。由于 RoPE 与这种吸收不兼容，位置信息走一条单独的“解耦 RoPE”分支，这条分支的键由所有头共享。

$$c_t^{KV} = W^{DKV} h_t, \quad k_t^C = W^{UK} c_t^{KV}, \quad v_t = W^{UV} c_t^{KV}, \quad k_t = [k_t^C;\ \mathrm{RoPE}(W^{KR} h_t)]$$

```
dc, dr = 16, 8                                          # 潜维度、RoPE 维度（远小于 2·H·d）
W_dkv = nn.Linear(D, dc, bias=False)                    # 下投影：cache 只存 c
W_uk, W_uv = nn.Linear(dc, H * d, bias=False), nn.Linear(dc, H * d, bias=False)
W_q, W_kr = nn.Linear(D, H * (d + dr), bias=False), nn.Linear(D, dr, bias=False)
c = W_dkv(x)                                            # (B,N,dc)  ← KV cache
k_c, v_m = split(W_uk(c)), split(W_uv(c))               # 上投影还原每头的 K、V
k_r = W_kr(x)[:, None].expand(B, H, N, dr)              # 解耦 RoPE 键，所有头共享（此处省略旋转）
qq = split(W_q(x))                                      # (B,H,N,d+dr)
out = sdpa(qq, torch.cat([k_c, k_r], -1), v_m, causal)
print(out.shape, "每 token 缓存:", dc + dr, "vs MHA:", 2 * H * d)
```

2.4 张量积注意力（Tensor Product Attention, TPA）

Zhang et al., 2025

把每个 token 的 $Q/K/V$（形状为 $H \times d$）分解成 R 个“头向量 ⊗ 维度向量”的秩一张量之和，cache 只存这些因子。它与 RoPE 天然兼容，MHA、MQA、GQA 都可以看作 TPA 的特例。

$$K_t = \frac{1}{R} \sum_{r=1}^{R} a_r(x_t) \otimes b_r(x_t), \qquad a_r \in \mathbb{R}^H,\ b_r \in \mathbb{R}^d$$

```
R = 2
Wa, Wb = nn.Linear(D, R * H), nn.Linear(D, R * d)
def tp(Wa, Wb, x):                                      # 返回 (B,H,N,d)
    a, b = Wa(x).view(B, N, R, H), Wb(x).view(B, N, R, d)   # cache 只存 a、b：R·(H+d) 个数
    return torch.einsum('bnrh,bnrd->bhnd', a, b) / R
k_tpa = tp(Wa, Wb, x)
v_tpa = tp(nn.Linear(D, R * H), nn.Linear(D, R * d), x)
out = sdpa(q, k_tpa, v_tpa, causal); print(out.shape)
```

2.5 跨层 KV 共享（CLA / YOCO）

Brandon et al., 2024（Cross-Layer Attention）；Sun et al., 2024（YOCO, You Only Cache Once）

相邻若干层共用同一份 KV cache，相当于在层的维度上做 MQA。YOCO 更进一步：前半部分网络用高效注意力生成一份全局 KV，后半部分所有层都对这份 KV 做交叉注意力，整个模型只缓存一次。

$$\mathrm{head}^{(\ell)} = \mathrm{Attn}\left(H^{(\ell)} W_Q^{(\ell)},\ K^{(s(\ell))},\ V^{(s(\ell))}\right), \qquad s(\ell) = \text{共享源层}$$

```
Wk, Wv = nn.Linear(D, D), nn.Linear(D, D)
Wq = nn.ModuleList(nn.Linear(D, D) for _ in range(3))
kv = split(Wk(x)), split(Wv(x))                         # 只在第 0 层计算并缓存
```

```
h = x
for l in range(3):                                          # 3 层共用同一份 KV
    h = h + merge(sdpa(split(Wq[l](h)), *kv, causal))
print(h.shape)
```

第 3 章 稀疏注意力

思路：每个查询只看一部分键，把 $O(N^2)$ 的复杂度降到 $O(N \cdot k)$。早期方法用**固定模式**决定看哪些键，2025 年以后的方法转向**可学习、可训练的动态选择**。

3.1 固定 / 跨步稀疏（Sparse Transformer）

Child et al., 2019 · OpenAI

用两种固定模式组合：局部窗口加上每隔 ℓ 取一个位置（strided），或者块内全连接加上每块的“汇总列”（fixed）。两层叠加后，任意两点之间都能传递信息。复杂度为 $O(N\sqrt{N})$。

$$A_i^{(1)} = \{j : i - j < \ell\}, \qquad A_i^{(2)} = \{j : (i - j) \bmod \ell = 0\}$$

```
l = 4
strided = causal & (((I - J) < l) | ((I - J) % l == 0))
fixed   = causal & ((I // l == J // l) | (J % l == l - 1))
out = sdpa(q, k, v, strided)
print(out.shape, "可见比例:", strided.float().mean().item())
```

3.2 滑动窗口注意力（Sliding Window Attention, SWA）

Beltagy et al., 2020；Mistral 7B, 2023；Gemma 2/3

每个 token 只看前面 w 个 token。堆叠 L 层后，感受野可以达到 $L \cdot w$。KV cache 固定为 w，可以用环形缓冲区实现。

$$M_{ij} = 0 \iff 0 \le i - j < w$$

```
w = 4
swa = causal & ((I - J) < w)
out = sdpa(q, k, v, swa); print(out.shape)
```

3.3 Longformer：窗口 + 全局 token

Beltagy et al., 2020

在滑动窗口的基础上，指定少数“全局 token”（如 [CLS]、问题部分），这些 token 能看所有位置，也能被所有位置看到。

$$M_{ij} = 0 \iff |i-j| \le w \ \vee \ i \in \mathcal{G} \ \vee \ j \in \mathcal{G}$$

```
g = torch.zeros(N, dtype=torch.bool); g[0] = True          # 第 0 个 token 为全局 token
mask = ((I - J).abs() <= 2) | g[None] | g[:, None]
out = sdpa(q, k, v, mask); print(out.shape)
```

3.4 BigBird：窗口 + 全局 + 随机

Zaheer et al., 2020

在 Longformer 的基础上再加随机连接。论文证明这三者组合仍然是图灵完备的，并且是序列函数的通用逼近器。

$$\mathcal{A}_i = \underbrace{\{|i-j| \le w\}}_{\text{窗口}} \cup \underbrace{\mathcal{G}}_{\text{全局}} \cup \underbrace{\mathcal{R}_i}_{\text{随机 } r \text{ 个}}$$

```
g = torch.zeros(N, dtype=torch.bool); g[0] = True
rand = torch.rand(N, N) < 0.1
mask = ((I - J).abs() <= 2) | g[None] | g[:, None] | rand
out = sdpa(q, k, v, mask); print(out.shape)
```

3.5 膨胀注意力（Dilated Attention, LongNet）

Ding et al., 2023 · 号称可扩展到 10 亿 token

把序列切成长度为 w 的段，段内每隔 r 个位置取一个参与注意力。多组 (w, r) 组合使用：小段、密集采样负责局部，大段、稀疏采样负责全局，复杂度为线性。

$$O = \sum_{(w,r)} \lambda_{(w,r)} \, \mathrm{Attn}(Q_{[w,r]}, K_{[w,r]}, V_{[w,r]})$$

```
def dilated(w, r): return (I // w == J // w) & ((I - J) % r == 0)
masks = [dilated(4, 1), dilated(8, 2), dilated(16, 4)]
out = sum(sdpa(q, k, v, m) for m in masks) / len(masks)    # 原文按各自 softmax 分母加权
print(out.shape)
```

3.6 LSH 注意力（Reformer）

Kitaev et al., 2020

用局部敏感哈希把相似的向量分进同一个桶，只在桶内计算注意力，复杂度为 $O(N \log N)$。前提是共享 $Q = K$。

$$h(x) = \arg\max\left([xR;\ -xR]\right), \qquad M_{ij} = 0 \iff h(q_i) = h(k_j)$$

```
nb = 4
qk = F.normalize(q, dim=-1)                              # Reformer 共享 Q=K
proj = qk @ torch.randn(d, nb // 2)
bucket = torch.cat([proj, -proj], -1).argmax(-1)         # (B,H,N)
mask = bucket[..., :, None] == bucket[..., None, :]
out = sdpa(qk, qk, v, mask); print(out.shape)
```

3.7 路由注意力（Routing Transformer）

Roy et al., 2021

用在线 k-means 对 Q、K 聚类，每个查询只看同一簇内的键，复杂度为 $O(N^{1.5})$。

$$M_{ij} = 0 \iff \arg\max_c \mu_c^\top q_i = \arg\max_c \mu_c^\top k_j$$

```
Cent = torch.randn(4, d)                                 # 聚类中心（训练中用 EMA 更新）
cq, ck = (q @ Cent.T).argmax(-1), (k @ Cent.T).argmax(-1)
mask = (cq[..., :, None] == ck[..., None, :]) | torch.eye(N, dtype=torch.bool)
out = sdpa(q, k, v, mask); print(out.shape)
```

3.8 注意力汇聚点与 StreamingLLM（Attention Sink）

Xiao et al., 2023

观察到的现象：模型会把大量注意力分给开头几个 token，称为“汇聚点”（sink），即使这些 token 语义上不重要。纯滑动窗口一旦把它们挤出 cache，模型就会崩溃。做法是始终保留开头 s 个 token，再加上最近的 w 个 token，这样就能做无限长的流式推理。

$$M_{ij} = 0 \iff j < s \ \vee\ 0 \le i - j < w$$

```
s, w = 2, 4
stream = causal & ((J < s) | ((I - J) < w))
out = sdpa(q, k, v, stream); print(out.shape)
```

3.9 原生稀疏注意力（Native Sparse Attention, NSA）

DeepSeek, 2025 · ACL 2025 最佳论文

与硬件对齐、端到端可训练的稀疏注意力。三条分支并行：

- **压缩**：把一块键压成一个，得到粗粒度的全局视野
- **选择**：用压缩分支的分数选出 top-n 块，在这些块上做细粒度注意力
- **滑窗**：负责局部

三路输出由门控加权求和。

$$o_t = \sum_{c\in\{\text{cmp},\text{slc},\text{win}\}} g_t^c \cdot \text{Attn}(q_t, \tilde{K}_t^c, \tilde{V}_t^c)$$

```
bs, topn, w = 4, 2, 4
Kc = k.view(B, H, N // bs, bs, d).mean(3)                # 压缩分支：块均值（原文为可学习 MLP）
Vc = v.view(B, H, N // bs, bs, d).mean(3)
o_cmp = sdpa(q, Kc, Vc)
p_blk = (q @ Kc.transpose(-2, -1)).softmax(-1)           # 复用压缩分数为每块打分
sel = p_blk.topk(topn, -1).indices
blk = torch.zeros(B, H, N, N // bs, dtype=torch.bool).scatter_(-1, sel, True)
o_slc = sdpa(q, k, v, blk.repeat_interleave(bs, -1))     # 选择分支：只看 top-n 块
o_win = sdpa(q, k, v, (I - J).abs() < w)                 # 滑窗分支
gate = torch.sigmoid(torch.randn(B, H, N, 3, 1))         # 原文由 MLP 从 x_t 产生
out = (gate * torch.stack([o_cmp, o_slc, o_win], 3)).sum(3)
print(out.shape)                                         #  （示例为非因果简化版）
```

3.10 块注意力混合（Mixture of Block Attention, MoBA）

Moonshot AI (Kimi), 2025

把 MoE 的思想用到注意力上：KV 按块切分，每块用均值池化后的 key 作为“专家表示”，查询只路由到 top-k 个块，当前块总是被选中。不引入新参数，可以在全注意力与稀疏注意力之间切换。

$$s_{t,b} = \langle q_t,\ \text{mean}(K_b)\rangle, \qquad \mathcal{B}_t = \text{TopK}_b(s_{t,b}), \qquad o_t = \text{Attn}(q_t, K_{\mathcal{B}_t}, V_{\mathcal{B}_t})$$

```
bs, topk = 4, 2
nblk = N // bs
score = q @ k.view(B, H, nblk, bs, d).mean(3).transpose(-2, -1)   # (B,H,N,nblk)
qblk, blk = torch.arange(N)[:, None] // bs, torch.arange(nblk)[None]
score = score.masked_fill(blk > qblk, float('-inf'))       # 不能选未来块
score = score.masked_fill(blk == qblk, float('inf'))        # 当前块必选
sel = score.topk(topk, -1).indices
bm = torch.zeros_like(score, dtype=torch.bool).scatter_(-1, sel, True)
out = sdpa(q, k, v, bm.repeat_interleave(bs, -1) & causal); print(out.shape)
```

3.11 DeepSeek 稀疏注意力（DeepSeek Sparse Attention, DSA）

DeepSeek-V3.2-Exp, 2025

用一个极轻量的“闪电索引器”（lightning indexer：头数少、维度小、可以跑在 FP8 上）为每个查询给所有历史 token 打分，选出 top-k 个 token，主注意力只在这 k 个 token 上计算。复杂度从 $O(N^2)$ 降到 $O(Nk)$。

$$I_{t,s} = \sum_{j=1}^{H^I} w_{t,j}^I \cdot \text{ReLU}\left(q_{t,j}^I \cdot k_s^I\right), \qquad o_t = \text{Attn}(q_t, \{k_s, v_s : I_{t,s} \in \text{TopK}(I_{t,:})\})$$

```
Hi, di, topk = 2, 8, 6                                  # 索引器头数、维度很小
qI, kI, wI = torch.randn(B, N, Hi, di), torch.randn(B, N, di), torch.randn(B, N, Hi)
logit = F.relu(torch.einsum('bthd,bsd->bths', qI, kI))
index = (wI[..., None] * logit).sum(2).masked_fill(~causal, float('-inf'))   # (B,N,N)
sel = index.topk(topk, -1).indices
mask = torch.zeros(B, N, N, dtype=torch.bool).scatter_(-1, sel, True) & causal
out = sdpa(q, k, v, mask[:, None]); print(out.shape)
```

3.12 稀疏注意力混合（Mixture of Sparse Attention, MoSA）

Piękos et al., 2025

采用 expert-choice 路由：每个注意力头自己挑出 k 个 token，只在这 k 个 token 内部做注意力，计算量为 $O(k^2)$。由于省下了算力，可以开更多的头。

$$\mathcal{I}_h = \mathrm{TopK}(XW_r^h), \qquad Y_h[\mathcal{I}_h] = \sigma(r_{\mathcal{I}_h}) \odot \mathrm{Attn}(Q_{\mathcal{I}_h}, K_{\mathcal{I}_h}, V_{\mathcal{I}_h})$$

```
kk = 6
r = torch.randn(B, H, N)                                # 路由分数（原文由 x 线性得到）
sel = r.topk(kk, -1).indices.sort(-1).values            # 每头选 kk 个 token，并保持时间顺序
idx = sel[..., None].expand(-1, -1, -1, d)
qs, ks, vs = q.gather(2, idx), k.gather(2, idx), v.gather(2, idx)
m = sel[..., :, None] >= sel[..., None, :]              # 子集内部保持因果
ys = sdpa(qs, ks, vs, m) * torch.sigmoid(r.gather(2, sel))[..., None]
out = torch.zeros_like(q).scatter(2, idx, ys); print(out.shape)
```

第 4 章 线性与亚二次注意力

核心技巧：去掉 softmax，换成可分解的核 $\phi(q)^\top\phi(k)$，再利用矩阵乘法结合律，把计算顺序从 $(QK^\top)V$ 改成 $Q(K^\top V)$，复杂度从 $O(N^2d)$ 降到 $O(Nd^2)$。在因果场景下，这等价于一个状态大小为 $d\times d$ 的 **RNN**。

2024 年以后的主线是给这个状态加上越来越精细的**遗忘门**和**更新规则**，演进路线为：加法 → 衰减 → 门控 → delta 规则 → 门控 delta → 测试时学习。

统一视角（状态 $S_t\in\mathbb{R}^{d_k\times d_v}$）：

方法	状态更新 $S_t=$	输出
线性注意力	$S_{t-1}+k_tv_t^\top$	$q_t^\top S_t$
RetNet	$\gamma S_{t-1}+k_tv_t^\top$	$q_t^\top S_t$
Mamba-2	$a_tS_{t-1}+k_tv_t^\top$	$q_t^\top S_t$
GLA	$\mathrm{Diag}(\alpha_t)S_{t-1}+k_tv_t^\top$	$q_t^\top S_t$
DeltaNet	$(I-\beta_tk_tk_t^\top)S_{t-1}+\beta_tk_tv_t^\top$	$q_t^\top S_t$
Gated DeltaNet	$\alpha_t(I-\beta_tk_tk_t^\top)S_{t-1}+\beta_tk_tv_t^\top$	$q_t^\top S_t$
KDA	$(I-\beta_tk_tk_t^\top)\mathrm{Diag}(\alpha_t)S_{t-1}+\beta_tk_tv_t^\top$	$q_t^\top S_t$

4.1 线性注意力（Linear Transformer）

Katharopoulos et al., 2020 · “Transformers are RNNs”

用特征映射 $\phi(x)=\mathrm{elu}(x)+1$（保证结果为正）代替 exp，利用结合律先算 $K^\top V$。在因果场景下，它就是一个累加状态的 RNN。

$$o_i=\frac{\phi(q_i)^\top\sum_{j\le i}\phi(k_j)v_j^\top}{\phi(q_i)^\top\sum_{j\le i}\phi(k_j)}=\frac{\phi(q_i)^\top S_i}{\phi(q_i)^\top z_i}$$

```
phi = lambda t: F.elu(t) + 1
Q, K = phi(q), phi(k)
out_nc = (Q @ (K.transpose(-2, -1) @ v)) / (Q @ K.sum(2, keepdim=True).transpose(-2, -1))  # 非因果 O(N)
S, z, ys = torch.zeros(B, H, d, d), torch.zeros(B, H, d), []
for t in range(N):                                      # 因果：RNN 形式
    S = S + K[:, :, t, :, None] * v[:, :, t, None, :]
```

```
    z = z + K[:, :, t]
    ys.append((Q[:, :, t, None] @ S).squeeze(2) / (Q[:, :, t] * z).sum(-1, keepdim=True))
out = torch.stack(ys, 2)
A = (Q @ K.transpose(-2, -1)) * causal                    # 对照：二次形式
print(torch.allclose(out, (A @ v) / A.sum(-1, keepdim=True), atol=1e-4))
```

4.2 Performer（FAVOR+）

Choromanski et al., 2021

用正随机特征无偏地近似 softmax 核，从而在线性复杂度下逼近真正的 softmax 注意力。原文使用正交随机矩阵 ω 来降低方差。

$$\exp(q^\top k) = \mathbb{E}_{\omega\sim\mathcal{N}(0,I)}\left[\phi(q)^\top\phi(k)\right], \qquad \phi(x) = \frac{1}{\sqrt{m}}\exp\left(\omega^\top x - \frac{\|x\|^2}{2}\right)$$

```
m = 1024                                                  # 随机特征数，越多越准
Wr = torch.randn(m, d)
def phi(t):
    t = t / d ** 0.25                                     # 吸收 1/√d 缩放
    return torch.exp(t @ Wr.T - (t ** 2).sum(-1, keepdim=True) / 2) / math.sqrt(m)
qs, ks = 0.3 * q, 0.3 * k                                 # 估计方差随 ‖q‖、‖k‖ 指数增长（实践中常配合
↪ QK-Norm）
Q, K = phi(qs), phi(ks)
out = (Q @ (K.transpose(-2, -1) @ v)) / (Q @ K.sum(2, keepdim=True).transpose(-2, -1))
ref = sdpa(qs, ks, v)
print("相对误差:", ((out - ref).norm() / ref.norm()).item())
```

4.3 Linformer

Wang et al., 2020

观察到注意力矩阵是低秩的，于是用投影矩阵 $E, F \in \mathbb{R}^{r\times N}$ 沿序列维把 K、V 压缩到 r 个位置。它不适合因果场景，也要求序列长度固定。

$$\text{Attn} = \text{softmax}\left(\frac{Q(EK)^\top}{\sqrt{d}}\right)(FV)$$

```
r = 8
E, Fp = torch.randn(r, N) / N ** .5, torch.randn(r, N) / N ** .5
out = sdpa(q, E @ k, Fp @ v)                              # 注意力矩阵只有 N×r
print(out.shape)
```

4.4 Nyströmformer

Xiong et al., 2021

用 Nyström 方法近似完整的注意力矩阵：选 m 个“地标”（取段均值），用三个小矩阵的乘积重建原矩阵。

$$\mathrm{softmax}\Big(\frac{QK^\top}{\sqrt{d}}\Big) \approx \mathrm{softmax}\Big(\frac{Q\tilde{K}^\top}{\sqrt{d}}\Big)\ \mathrm{softmax}\Big(\frac{\tilde{Q}\tilde{K}^\top}{\sqrt{d}}\Big)^{+}\ \mathrm{softmax}\Big(\frac{\tilde{Q}K^\top}{\sqrt{d}}\Big)$$

```
m = 4
ql, kl = q.view(B, H, m, N // m, d).mean(3), k.view(B, H, m, N // m, d).mean(3)   # 地标
sm = lambda a, b: (a @ b.transpose(-2, -1) / math.sqrt(d)).softmax(-1)
out = sm(q, kl) @ torch.linalg.pinv(sm(ql, kl)) @ (sm(ql, k) @ v)   # 原文用迭代法求伪逆
print(out.shape)
```

4.5 cosFormer

Qin et al., 2022

用 ReLU 作为特征映射，再乘上 cos 形式的距离重加权，引入局部性偏好。利用 $\cos(a-b)=\cos a\cos b+\sin a\sin b$ 可以拆开计算，保持线性复杂度。

$$s_{ij}=\mathrm{ReLU}(q_i)^\top \mathrm{ReLU}(k_j)\cos\Big(\frac{\pi}{2}\cdot\frac{i-j}{M}\Big)$$

```
th = math.pi / 2 * torch.arange(N) / N
Q, K = F.relu(q), F.relu(k)
Qc, Qs, Kc, Ks = Q * th.cos()[:, None], Q * th.sin()[:, None], K * th.cos()[:, None], K * th.sin()[:, None]
num = Qc @ (Kc.transpose(-2, -1) @ v) + Qs @ (Ks.transpose(-2, -1) @ v)
den = Qc @ Kc.sum(2, keepdim=True).transpose(-2, -1) + Qs @ Ks.sum(2, keepdim=True).transpose(-2, -1)
out = num / (den + 1e-6); print(out.shape)
```

4.6 Hedgehog

Zhang et al., 2024 · ICLR

用可学习的特征映射 $\phi(x)=[\mathrm{softmax}(Wx);\ \mathrm{softmax}(-Wx)]$，通过蒸馏训练，让线性注意力的注意力图去模仿 softmax 注意力的“尖峰”和“单调”特性。适合把已经训练好的 Transformer 转换成线性注意力模型。

$$\phi(x)=\big[\mathrm{softmax}(Wx)\,\|\,\mathrm{softmax}(-Wx)\big],\qquad \mathcal{L}=\mathrm{CE}\big(\mathrm{softmax}(QK^\top),\ \hat{A}_\phi\big)$$

```
Wf = nn.Linear(d, d)
phi = lambda t: torch.cat([Wf(t).softmax(-1), (-Wf(t)).softmax(-1)], -1)
Q, K = phi(q), phi(k)
A_lin = Q @ K.transpose(-2, -1); A_lin = A_lin / A_lin.sum(-1, keepdim=True)
A_soft = (q @ k.transpose(-2, -1) / math.sqrt(d)).softmax(-1)
loss = -(A_soft * A_lin.log()).sum(-1).mean()              # 蒸馏损失：训练 Wf 使 A_lin ≈ A_soft
out = A_lin @ v; print(out.shape, loss.item())
```

4.7 Based

Arora et al., 2024 · Hazy Research

用二阶泰勒展开 $\exp(x) \approx 1 + x + x^2/2$ 作为特征映射，近似 softmax 的全局部分，再配一个很小的精确 softmax 滑窗来补足局部精度。它在“召回–吞吐”的权衡上表现很好。

$$\phi(x) = \left[1,\ x,\ \frac{x \otimes x}{\sqrt{2}}\right] \ \Rightarrow\ \phi(q)^\top \phi(k) = 1 + q^\top k + \frac{(q^\top k)^2}{2}$$

```
dt = 8
Wp = nn.Linear(d, dt, bias=False)
def phi(t):
    t = Wp(t) / dt ** 0.25
    return torch.cat([torch.ones_like(t[..., :1]), t, (t[..., :, None] * t[..., None, :]).flatten(-2) /
    ↪  math.sqrt(2)], -1)
Q, K = phi(q), phi(k)
A = (Q @ K.transpose(-2, -1)) * causal                    # 二次形式展示；实际按线性递推计算
lin = (A @ v) / A.sum(-1, keepdim=True)
win = sdpa(q, k, v, causal & (I - J < 4))                 # 小窗口精确 softmax
out = lin + win; print(out.shape)
```

4.8 保持机制（Retention, RetNet）

Sun et al., 2023 · 微软

给线性注意力的状态加上指数衰减 γ，并且每个头使用不同的 γ（多尺度）。同一个机制有三种等价的计算形式：并行（训练用）、递归（推理用，$O(1)$ 每步）、分块（长序列用）。

$$S_n = \gamma S_{n-1} + k_n^\top v_n,\quad o_n = q_n S_n \qquad \Longleftrightarrow \qquad O = (QK^\top \odot \mathcal{D})V,\ \mathcal{D}_{nm} = \gamma^{n-m}\mathbf{1}[n \geq m]$$

```
gamma = 1 - 2.0 ** (-5 - torch.arange(H).float())          # 每头不同的衰减
Dm = (gamma[:, None, None] ** (I - J).clamp(min=0)) * causal
out_par = (q @ k.transpose(-2, -1) * Dm) @ v               # 并行形式
S, ys = torch.zeros(B, H, d, d), []
for t in range(N):                                         # 递归形式
    S = gamma[:, None, None] * S + k[:, :, t, :, None] * v[:, :, t, None, :]
    ys.append((q[:, :, t, None] @ S).squeeze(2))
out = torch.stack(ys, 2)
print(torch.allclose(out_par, out, rtol=1e-4, atol=1e-3))
```

4.9 RWKV（WKV 注意力）

Peng et al., 2023 · RWKV-4；后续 RWKV-5/6/7 依次引入矩阵状态、数据依赖衰减、广义 delta 规则

按通道衰减的指数加权平均，可以写成 RNN 形式，也可以并行训练。u 是给当前 token 的额外“奖励”。

$$wkv_t = \frac{\sum_{i<t} e^{-(t-1-i)w + k_i} v_i + e^{u + k_t} v_t}{\sum_{i<t} e^{-(t-1-i)w + k_i} + e^{u + k_t}}$$

```
C = D
w, u = torch.rand(C), torch.zeros(C)                       # 每通道衰减、当前 token 奖励
kr, vr, rr = torch.randn(B, N, C), torch.randn(B, N, C), torch.randn(B, N, C)
```

```
a, b, ys = torch.zeros(B, C), torch.zeros(B, C), []
for t in range(N):
    e = torch.exp(u + kr[:, t])
    ys.append((a + e * vr[:, t]) / (b + e))
    a = torch.exp(-w) * a + torch.exp(kr[:, t]) * vr[:, t]
    b = torch.exp(-w) * b + torch.exp(kr[:, t])
out = torch.sigmoid(rr) * torch.stack(ys, 1)              # receptance 门
print(out.shape)
```

4.10 门控线性注意力（Gated Linear Attention, GLA）

Yang et al., 2024 · ICML

给状态加上**数据依赖**、**逐维度**的遗忘门 α_t，同时提供硬件高效的分块并行算法。与之同族的还有 HGRN2、Mamba-1 的选择性机制等。

$$S_t = \mathrm{Diag}(\alpha_t)\, S_{t-1} + k_t v_t^\top, \qquad o_t = S_t^\top q_t, \qquad \alpha_t = \sigma(x_t W_\alpha)^{1/\tau}$$

```
alpha = torch.sigmoid(torch.randn(B, H, N, d))            # 原文由 x 经低秩投影得到
S, ys = torch.zeros(B, H, d, d), []
for t in range(N):
    S = alpha[:, :, t, :, None] * S + k[:, :, t, :, None] * v[:, :, t, None, :]
    ys.append((q[:, :, t, None] @ S).squeeze(2))
out = torch.stack(ys, 2); print(out.shape)
```

4.11 DeltaNet 与 Gated DeltaNet

Schlag et al., 2021；Yang et al., 2024（DeltaNet 并行化）；Yang et al., 2025（Gated DeltaNet, ICLR）· Qwen3-Next 使用

线性注意力只会“往状态里加”，容量满了之后检索会变模糊。**Delta 规则**改为先读出 k_t 对应的旧值，再朝目标值 v_t 修正，相当于对 $\|S^\top k_t - v_t\|^2$ 做一步在线梯度下降。Gated DeltaNet 在此基础上再乘一个标量衰减 α_t，可以快速清空记忆。

$$\text{DeltaNet: } S_t = (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top \qquad \text{Gated: } S_t = \alpha_t (I - \beta_t k_t k_t^\top) S_{t-1} + \beta_t k_t v_t^\top$$

```
kn = F.normalize(k, dim=-1)                               # key 需 L2 归一化
beta = torch.sigmoid(torch.randn(B, H, N, 1, 1))          # 写入强度
alpha = torch.sigmoid(torch.randn(B, H, N, 1, 1))         # 衰减门；全取 1 即为 DeltaNet
S, ys = torch.zeros(B, H, d, d), []
for t in range(N):
    kt, vt = kn[:, :, t, :, None], v[:, :, t, :, None]    # (B,H,d,1)
    S = alpha[:, :, t] * (S - beta[:, :, t] * kt @ (kt.transpose(-2, -1) @ S)) \
        + beta[:, :, t] * kt @ vt.transpose(-2, -1)
    ys.append((q[:, :, t, None] @ S).squeeze(2))
out = torch.stack(ys, 2); print(out.shape)
```

4.12 闪电注意力（Lightning Attention）

Qin et al., 2024 · MiniMax-01 使用

这是线性注意力的一种 IO 感知分块实现。块内用“左乘”（$QK^\top$ 加掩码）计算，块间用“右乘”（累计状态 $K^\top V$）计算，避开了因果线性注意力里低效的 cumsum，使训练速度不随序列长度下降。它来自 TransNormer 系列：分母去掉，改为在输出端做归一化。

$$O_b = \underbrace{[(Q_b K_b^\top) \odot M] V_b}_{\text{块内}} + \underbrace{Q_b\, S_{b-1}}_{\text{块间}}, \qquad S_b = S_{b-1} + K_b^\top V_b$$

```
bs, outs = 4, []
S, mb = torch.zeros(B, H, d, d), torch.ones(bs, bs).tril()
for s0 in range(0, N, bs):
    qb, kb, vb = q[:, :, s0:s0 + bs], k[:, :, s0:s0 + bs], v[:, :, s0:s0 + bs]
    outs.append(((qb @ kb.transpose(-2, -1)) * mb) @ vb + qb @ S)   # 块内左乘 + 块间右乘
    S = S + kb.transpose(-2, -1) @ vb
out = torch.cat(outs, 2)
print(torch.allclose(out, ((q @ k.transpose(-2, -1)) * causal) @ v, rtol=1e-4, atol=1e-3))
```

4.13 Kimi Delta Attention（KDA）

Moonshot AI, 2025 · Kimi Linear

把 Gated DeltaNet 的标量衰减改为**逐通道**衰减（类似 GLA 的细粒度门控），用来更精细地控制有限 RNN 状态的记忆。Kimi Linear 按 3:1 的比例交错 KDA 层与 MLA 层，在长上下文任务上超过了全注意力基线。

$$S_t = (I - \beta_t k_t k_t^\top)\,\mathrm{Diag}(\alpha_t)\, S_{t-1} + \beta_t k_t v_t^\top$$

```
kn = F.normalize(k, dim=-1)
beta = torch.sigmoid(torch.randn(B, H, N, 1, 1))
alpha = torch.sigmoid(torch.randn(B, H, N, d, 1))          # 逐通道遗忘门（GDN 是标量）
S, ys = torch.zeros(B, H, d, d), []
for t in range(N):
    kt, vt = kn[:, :, t, :, None], v[:, :, t, :, None]
    S = alpha[:, :, t] * S                                  # 先做细粒度衰减
    S = S - beta[:, :, t] * kt @ (kt.transpose(-2, -1) @ S) + beta[:, :, t] * kt @ vt.transpose(-2, -1)
    ys.append((q[:, :, t, None] @ S).squeeze(2))
out = torch.stack(ys, 2); print(out.shape)
```

4.14 状态空间对偶（Mamba-2 / SSD）

Dao & Gu, 2024 · ICML

证明当状态空间模型（SSM）的转移矩阵是“标量乘单位阵”时，SSM 等价于一种带掩码 L 的线性注意力。对应关系为 $C \leftrightarrow Q$，$B \leftrightarrow K$，$X \leftrightarrow V$。这一结论把 Mamba 与注意力统一到“结构化掩码注意力”的框架下。

$$h_t = a_t h_{t-1} + B_t x_t^\top,\ y_t = C_t^\top h_t \quad\Longleftrightarrow\quad Y = (L \odot CB^\top)X,\ L_{ij} = \prod_{k=j+1}^{i} a_k$$

```
ds = 8
Cm, Bm, X = torch.randn(B, N, ds), torch.randn(B, N, ds), torch.randn(B, N, d)
a = torch.sigmoid(torch.randn(B, N))
la = torch.log(a).cumsum(-1)
L = torch.exp((la[:, :, None] - la[:, None, :]).masked_fill(~causal, float('-inf')))
y_attn = ((Cm @ Bm.transpose(1, 2)) * L) @ X              # 注意力（二次）形式
h, ys = torch.zeros(B, ds, d), []
for t in range(N):                                        # SSM（递归）形式
    h = a[:, t, None, None] * h + Bm[:, t, :, None] * X[:, t, None, :]
    ys.append((Cm[:, t, None] @ h).squeeze(1))
print(torch.allclose(y_attn, torch.stack(ys, 1), rtol=1e-4, atol=1e-4))
```

4.15 对数线性注意力（Log-Linear Attention）

Guo et al., 2025

介于线性注意力（状态大小固定）与 softmax 注意力（KV 随长度线性增长）之间。它按 Fenwick 树把历史切成 $O(\log N)$ 个桶，每个桶维护一个状态，每层再配一个数据依赖的权重 λ。时间复杂度为 $O(N \log N)$，状态只随长度对数增长。可以套在 Mamba-2、Gated DeltaNet 等模型上使用。

$$o_t = \sum_{\ell=0}^{L-1} \lambda_t^{(\ell)}\, q_t^\top S_t^{(\ell)} \qquad \Longleftrightarrow \qquad O = (QK^\top \odot M^{\mathcal{H}})V,\ \ M^{\mathcal{H}}_{ij} = \lambda_i^{\ell(i,j)}$$

```
lvl = torch.where(I == J, 0, torch.floor(torch.log2((I ^ J).float().clamp(min=1)))).long() + 1  # 层级（简化）
Lv = int(lvl.max()) + 1                                   # 层数 = O(log N)
lam = torch.rand(B, H, N, Lv)                             # 每个查询对每层的权重（由 x 产生）
Mh = lam.gather(-1, lvl.expand(B, H, N, N)) * causal
out = (q @ k.transpose(-2, -1) * Mh) @ v                  # 二次形式展示；实际为分块 O(N log N)
print(out.shape, "层数:", Lv)
```

4.16 测试时训练层（Test-Time Training, TTT）

Sun et al., 2024

把 RNN 的隐状态换成一个**模型的权重** W，每读一个 token，就对自监督损失做一步梯度下降来更新 W。当 W 是线性层时（TTT-Linear），这个更新恰好就是 delta 规则；W 也可以是 MLP（TTT-MLP），表达能力更强。

$$\ell(W; x_t) = \| W^\top \theta_K x_t - \theta_V x_t \|^2, \qquad W_t = W_{t-1} - \eta \nabla \ell(W_{t-1}; x_t), \qquad z_t = W_t^\top \theta_Q x_t$$

```
eta = 0.1
W, ys = torch.zeros(B, H, d, d), []
for t in range(N):
    kt, vt, qt = k[:, :, t], v[:, :, t], q[:, :, t]       # θ_K x、θ_V x、θ_Q x
    err = (kt[..., None, :] @ W).squeeze(-2) - vt         # 预测 − 目标
    W = W - eta * kt[..., :, None] * err[..., None, :]    # 一步梯度下降 = 状态更新
    ys.append((qt[..., None, :] @ W).squeeze(-2))
out = torch.stack(ys, 2); print(out.shape)
```

4.17 Titans：神经长期记忆

Behrouz et al., 2025 · Google

把长期记忆做成一个在测试时学习的深度网络（MLP），更新时引入两项机制：

- **动量**：“惊讶度”随时间累积
- **权重衰减**：对应遗忘

这个记忆模块与短期的注意力组合使用，有 MAC、MAG、MAL 三种接法。

$$S_t = \eta_t S_{t-1} - \theta_t \nabla \ell(M_{t-1}; x_t), \qquad M_t = (1-\alpha_t) M_{t-1} + S_t$$

```
eta, theta, alpha = 0.9, 0.1, 0.01                          # 动量、学习率、遗忘率（原文都是数据依赖的）
M, Sm, ys = torch.zeros(B, H, d, d), torch.zeros(B, H, d, d), []
for t in range(N):                                          # 这里用线性记忆，原文为深层 MLP
    kt, vt = k[:, :, t], v[:, :, t]
    grad = kt[..., :, None] * ((kt[..., None, :] @ M).squeeze(-2) - vt)[..., None, :]
    Sm = eta * Sm - theta * grad                            # 惊讶度（带动量）
    M = (1 - alpha) * M + Sm                                # 带遗忘的记忆更新
    ys.append((q[:, :, t, None, :] @ M).squeeze(-2))
out = torch.stack(ys, 2); print(out.shape)
```

第 5 章 对注意力本身的改造

保持 $O(N^2)$ 的全注意力不变，只修改打分函数、归一化方式、输出门控或头之间的交互，目的是提升质量、稳定训练、减少“注意力汇聚点”和大幅激活值（massive activations）。

5.1 差分注意力（Differential Attention, DIFF Transformer）

Ye et al., 2024 · ICLR 2025 · 微软

把两张 softmax 注意力图相减，抵消共同的“噪声注意力”，类似差分放大器的原理。这样能减少幻觉，提升长上下文检索能力。

$$\mathrm{DiffAttn}(X) = \left[\mathrm{softmax}\left(\frac{Q_1 K_1^\top}{\sqrt{d}}\right) - \lambda\,\mathrm{softmax}\left(\frac{Q_2 K_2^\top}{\sqrt{d}}\right)\right] V, \quad \lambda = e^{\lambda_{q1}\cdot\lambda_{k1}} - e^{\lambda_{q2}\cdot\lambda_{k2}} + \lambda_{\text{init}}$$

```
q1, q2 = q.chunk(2, -1); k1, k2 = k.chunk(2, -1)
lam_init, lam = 0.8, 0.5                                  # λ 原文为可学习重参数化
A1 = (q1 @ k1.transpose(-2, -1) / math.sqrt(d // 2)).softmax(-1)
A2 = (q2 @ k2.transpose(-2, -1) / math.sqrt(d // 2)).softmax(-1)
out = (A1 - lam * A2) @ v
out = F.rms_norm(out, (d,)) * (1 - lam_init)              # 每头独立归一化
print(out.shape)
```

5.2 门控注意力（Gated Attention）

Qiu et al., 2025 · Qwen 团队 · NeurIPS 2025 最佳论文 · Qwen3-Next 使用

在 SDPA 输出之后、W_O 之前，乘一个由当前 token 产生的、逐头逐维的 sigmoid 门。这个简单改动带来了非线性与稀疏性，几乎消除了 attention sink 和大幅激活值，同时提升训练稳定性和长度外推能力。

$$Y' = \mathrm{SDPA}(Q, K, V) \odot \sigma(XW_\theta), \qquad O = Y'W_O$$

```
Wg = nn.Linear(D, D)
y = sdpa(q, k, v, causal)                                 # (B,H,N,d)
gate = torch.sigmoid(split(Wg(x)))                        # 由当前 token 产生，逐头逐维
out = merge(y * gate)                                     # 之后再接 W_O
print(out.shape)
```

5.3 Sigmoid 注意力

Ramapuram et al., 2024 · Apple · ICLR 2025

用逐元素 sigmoid 代替 softmax，并加上偏置 $b = -\log N$ 防止早期训练不稳定。由于去掉了行归一化，各位置之间不再“争夺”注意力，kernel（FlashSigmoid）也更快。

$$\text{SigmoidAttn}(Q,K,V) = \sigma\left(\frac{QK^\top}{\sqrt{d}} + b\right)V, \qquad b = -\log N$$

```
s = q @ k.transpose(-2, -1) / math.sqrt(d) - math.log(N)
out = torch.sigmoid(s.masked_fill(~causal, float('-inf'))) @ v
print(out.shape)
```

5.4 Softpick

Zuhri et al., 2025

一种可以直接替换 softmax 的归一化函数，它的输出不强制和为 1，可以全为 0。分子用 $\text{ReLU}(e^x - 1)$，使负分数被精确置零。这样就消除了 attention sink 和大幅激活值，对低比特量化更友好。

$$\text{softpick}(x)_i = \frac{\text{ReLU}(e^{x_i} - 1)}{\sum_j |e^{x_j} - 1|}$$

```
def softpick(s, eps=1e-6):
    m = s.amax(-1, keepdim=True).clamp(min=0)              # 数值稳定：分子分母同除 e^m
    e = torch.exp(s - m) - torch.exp(-m)
    return F.relu(e) / (e.abs().sum(-1, keepdim=True) + eps)
A = softpick(q @ k.transpose(-2, -1) / math.sqrt(d))
out = A @ v
print(out.shape, "精确为 0 的比例:", (A == 0).float().mean().item())
```

5.5 Off-by-One Softmax 与可学习 Sink

Miller, 2023（softmax_1）；OpenAI gpt-oss, 2025（每头可学习的 sink logit）

在 softmax 分母上加一个“空选项”，允许注意力“什么都不看”。这等价于在序列里加一个打分为 0（或可学习）、值为 0 的虚拟 token。

$$\text{softmax}_1(x)_i = \frac{e^{x_i}}{1 + \sum_j e^{x_j}}, \qquad \text{一般形式：}\ \frac{e^{x_i}}{e^{s_h} + \sum_j e^{x_j}}$$

```
s = q @ k.transpose(-2, -1) / math.sqrt(d)
sink = torch.zeros(1, H, 1, 1)                             # gpt-oss：每头一个可学习标量；取 0 即 softmax_1
A = torch.cat([s, sink.expand(B, H, N, 1)], -1).softmax(-1)[..., :-1]   # sink 只吸收概率，不贡献值
out = A @ v
print(out.shape, "行和 < 1:", (A.sum(-1) < 1).all().item())
```

5.6 遗忘 Transformer（Forgetting Transformer, FoX）

Lin et al., 2025 · ICLR

把 RNN 的数据依赖遗忘门引入 softmax 注意力：在打分上加一个累积的 $\log f$ 偏置，越早的 token 被遗忘得越多。不需要位置编码，可以直接使用 FlashAttention 实现。

$$O = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}} + D\right)V, \qquad D_{ij} = \sum_{l=j+1}^{i} \log f_l, \quad f_t = \sigma(w_f^\top x_t + b_f)$$

```
logf = F.logsigmoid(torch.randn(B, H, N))                # 每头每步的遗忘门（由 x 产生）
c = logf.cumsum(-1)
Dmat = c[..., :, None] - c[..., None, :]                 # Σ_{l=j+1..i} log f_l
out = sdpa(q, k, v, causal, bias=Dmat); print(out.shape)
```

5.7 Stick-Breaking 注意力

Tan et al., 2025 · ICLR · IBM

用“折棍过程”代替 softmax：从最近的 token 开始依次分配注意力，每个 token 拿走剩余份额的 β，更远的 token 只能分到剩下的部分。这样天然带有“近因偏好”，不需要位置编码，长度泛化能力好。

$$\beta_{ij} = \sigma(q_i^\top k_j), \qquad A_{ij} = \beta_{ij} \prod_{j<k<i} (1 - \beta_{ik})$$

```
z = q @ k.transpose(-2, -1) / math.sqrt(d)
logb, log1mb = F.logsigmoid(z), F.logsigmoid(-z).masked_fill(~causal, 0)
rev = log1mb.flip(-1).cumsum(-1).flip(-1) - log1mb        # Σ_{k>j} log(1-β_ik)（此处含自身 k=i）
A = torch.exp(logb + rev).masked_fill(~causal, 0)
out = A @ v
print(out.shape, "行和 ⩽ 1:", (A.sum(-1) <= 1 + 1e-5).all().item())
```

5.8 多 Token 注意力（Multi-Token Attention, MTA）

Golovneva et al., 2025 · Meta

标准注意力的每个权重只由一对 (q_i, k_j) 决定。MTA 在注意力打分上做二维卷积（同时沿查询和键方向），让相邻的多个 query/key 共同决定权重，从而能定位“同时出现多个关键词”的位置。另外还在头之间做卷积混合。

$$\hat{A} = \mathrm{softmax}\Big(\mathrm{Conv2d}_\theta\big(\tfrac{QK^\top}{\sqrt{d}}\big)\Big), \qquad \tilde{A}_h = \sum_{h'} w_{hh'} \hat{A}_{h'}$$

```
s = (q @ k.transpose(-2, -1) / math.sqrt(d)).masked_fill(~causal, 0)   # 卷积前先把未来位置置 0
conv = nn.Conv2d(H, H, (3, 3), groups=H)                  # 每头一个 3×3 "键-查询卷积"
s = conv(F.pad(s, (1, 1, 2, 0))).masked_fill(~causal, float('-inf'))  # 查询维只向过去看
A = torch.einsum('gh,bhij->bgij', torch.randn(H, H).softmax(-1), s.softmax(-1))  # 头间混合
out = A @ v; print(out.shape)
```

5.9 Talking-Heads 注意力

Shazeer et al., 2020

在 softmax 之前和之后各加一个 $H \times H$ 的线性变换，让不同头的注意力打分互相“交流”。

$$\tilde{S}_g = \sum_h P^{(l)}_{hg} S_h, \qquad \tilde{A}_g = \sum_h P^{(w)}_{hg} \,\mathrm{softmax}(\tilde{S}_h)$$

```
Pl, Pw = torch.randn(H, H) / H ** .5, torch.randn(H, H) / H ** .5
s = torch.einsum('hg,bhij->bgij', Pl, q @ k.transpose(-2, -1) / math.sqrt(d))   # softmax 前混合
A = torch.einsum('hg,bhij->bgij', Pw, s.softmax(-1))                            # softmax 后混合
out = A @ v; print(out.shape)
```

5.10 注意力头混合（MoA / MoH）

Zhang et al., 2022（Mixture of Attention Heads）；Jin et al., 2024（MoH）

把每个注意力头当作一个“专家”，由路由器为每个 token 选出 top-k 个头并加权组合。MoH 还会保留少数始终激活的共享头。

$$O_t = \sum_{h \in \mathrm{TopK}(g_t)} g_{t,h}\, \mathrm{head}_h(t)\, W^h_O, \qquad g_t = \mathrm{softmax}(W_r x_t)$$

```
router = nn.Linear(D, H)
heads = sdpa(q, k, v, causal)                          # (B,H,N,d)
g = router(x).softmax(-1)                              # (B,N,H)
topv, topi = g.topk(2, -1)
w = torch.zeros_like(g).scatter(-1, topi, topv)        # 每个 token 只激活 2 个头
out = merge(heads * w.transpose(1, 2)[..., None])      # 加权后拼接，再接 W_O
print(out.shape)
```

5.11 QK 归一化（QK-Norm）

Henry et al., 2020；Dehghani et al., 2023（ViT-22B）· OLMo 2、Gemma 3、Qwen3 等使用

在点积之前对 q、k 分别做 LayerNorm 或 RMSNorm，防止注意力打分（logits）无限增长、导致训练发散。这是大模型训练中常见的稳定性技巧。

$$s_{ij} = \frac{\mathrm{Norm}(q_i)^\top \mathrm{Norm}(k_j)}{\sqrt{d}}$$

```
gq, gk = torch.ones(d), torch.ones(d)                  # 可学习缩放
qn, kn = F.rms_norm(q, (d,), gq), F.rms_norm(k, (d,), gk)
out = sdpa(qn, kn, v, causal); print(out.shape)
```

5.12 选择性注意力（Selective Attention）

Leviathan et al., 2024 · Google

无参数的改动：让一个 token 可以“宣布”某个更早的 token 已经没用了，之后的所有 token 都会减少对它的注意力。被屏蔽的 token 还可以从 KV cache 中删除，节省显存。

$$S = \mathrm{ReLU}(\text{某个头的打分}), \qquad F_{ij} = \sum_{k<i} S_{kj}, \qquad A = \mathrm{softmax}\Big(\frac{QK^\top}{\sqrt{d}} - F\Big)$$

```
z = q @ k.transpose(-2, -1) / math.sqrt(d)
Ssel = F.relu(z[:, :1]).masked_fill(~causal, 0) * (1 - torch.eye(N))   # 用第 0 头打分作为"屏蔽"信号
Fm = Ssel.cumsum(-2) - Ssel                                # 截至 i-1 的累计屏蔽量
out = sdpa(q, k, v, causal, bias=-Fm); print(out.shape)
```

5.13 可扩展 Softmax（Scalable-Softmax, SSMax）

Nakanishi, 2025

上下文变长后，softmax 分布会被“摊平”，注意力无法聚焦。SSMax 把打分乘上 $s \log n$（n 为当前可见的 token 数），保证不同长度下的聚焦能力一致，改善长度外推。Llama 4 中的注意力温度调节思路与此类似。

$$\mathrm{SSMax}(z)_i = \frac{n^{sz_i}}{\sum_j n^{sz_j}} = \mathrm{softmax}(s \log n \cdot z)_i$$

```
s_param = 0.43                                             # 每头可学习
n = torch.arange(1, N + 1).float()[:, None]                # 第 i 个查询可见 i+1 个 token
z = q @ k.transpose(-2, -1) / math.sqrt(d)
out = (s_param * torch.log(n) * z).masked_fill(~causal, float('-inf')).softmax(-1) @ v
print(out.shape)
```

第 6 章 位置信息

注意力本身对输入的排列顺序不敏感（置换等变），位置信息必须额外注入。注入的位置有三种：输入端（绝对位置编码）、打分偏置（相对位置、ALiBi），以及对 q、k 的旋转（RoPE、PaTH）。

6.1 正弦绝对位置编码（Sinusoidal）

Vaswani et al., 2017

在输入端加上固定的正余弦向量，不同维度对应不同的频率。

$$PE_{(p,2i)} = \sin\left(p/10000^{2i/D}\right), \qquad PE_{(p,2i+1)} = \cos\left(p/10000^{2i/D}\right)$$

```
p = torch.arange(N).float()[:, None]
inv = 10000 ** (-torch.arange(0, D, 2).float() / D)
pe = torch.zeros(N, D); pe[:, 0::2], pe[:, 1::2] = torch.sin(p * inv), torch.cos(p * inv)
out = x + pe                                                    # 加到输入嵌入上
print(out.shape)
```

6.2 相对位置表示（Shaw 相对位置）

Shaw et al., 2018

在打分中加入“查询与相对距离嵌入”的点积，距离超过一定范围后截断。Transformer-XL 后来把它扩展为 4 项分解。

$$e_{ij} = \frac{q_i^\top \left(k_j + a^K_{\mathrm{clip}(j-i,\,-c,\,c)}\right)}{\sqrt{d}}$$

```
c = 4
rel = (J - I).clamp(-c, c) + c                                  # 相对距离 → 下标
aK = nn.Embedding(2 * c + 1, d)(rel)                            # (N,N,d)
s = (q @ k.transpose(-2, -1) + torch.einsum('bhid,ijd->bhij', q, aK)) / math.sqrt(d)
out = s.softmax(-1) @ v; print(out.shape)
```

6.3 T5 相对位置偏置

Raffel et al., 2020

更简单的做法：每个头为每个“相对距离桶”学习一个标量偏置，直接加到打分上。距离越远，分桶越粗（对数分桶）。

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d}} + b_h(\text{bucket}(j-i))$$

```
rel = J - I
sign, a = (rel > 0).long(), rel.abs()
bucket = torch.where(a < 4, a, 4 + (torch.log(a.float().clamp(min=1) / 4) / math.log(32 / 4) *
↪  4).long().clamp(max=3))
bucket = bucket + sign * 8                                  # 正负方向各 8 个桶
bias = nn.Embedding(16, H)(bucket).permute(2, 0, 1)         # (H,N,N)
out = sdpa(q, k, v, bias=bias); print(out.shape)
```

6.4 ALiBi（线性偏置注意力）

Press et al., 2022 · BLOOM、MPT 使用

不使用任何位置嵌入，只在打分上减去一个与距离成正比的惩罚，每个头的斜率 m_h 不同（几何级数）。这样做长度外推能力强。

$$s_{ij} = \frac{q_i^\top k_j}{\sqrt{d}} - m_h\,(i-j), \qquad m_h = 2^{-8h/H}$$

```
m = 2.0 ** (-8 * torch.arange(1, H + 1) / H)
bias = -m[:, None, None] * (I - J).clamp(min=0)             # (H,N,N)
out = sdpa(q, k, v, causal, bias); print(out.shape)
```

6.5 旋转位置编码（RoPE）

Su et al., 2021 · 几乎所有现代 LLM 使用

把 q、k 的每两维看成一个复数，按位置 p 旋转角度 $p\theta_i$。这样内积只依赖相对位置：$\langle R_m q, R_n k\rangle = \langle q, R_{n-m} k\rangle$。

$$\text{RoPE}(x,p)_{[2i:2i+2]} = \begin{pmatrix}\cos p\theta_i & -\sin p\theta_i \\ \sin p\theta_i & \cos p\theta_i\end{pmatrix} x_{[2i:2i+2]}, \qquad \theta_i = \text{base}^{-2i/d}$$

```
def rope(t, base=10000.0):
    pos = torch.arange(t.size(-2)).float()
    ang = pos[:, None] * base ** (-torch.arange(0, d, 2).float() / d)   # (N,d/2)
    t1, t2 = t[..., ::2], t[..., 1::2]
    return torch.stack([t1 * ang.cos() - t2 * ang.sin(), t1 * ang.sin() + t2 * ang.cos()], -1).flatten(-2)
out = sdpa(rope(q), rope(k), v, causal)
qa = rope(q[..., :1, :].expand(-1, -1, N, -1)); ka = rope(k[..., :1, :].expand(-1, -1, N, -1))
```

```
print(torch.allclose((qa[..., 3, :] * ka[..., 1, :]).sum(-1), (qa[..., 5, :] * ka[..., 3, :]).sum(-1),
↪  atol=1e-4))  # 只依赖相对距离
```

6.6 RoPE 长度外推：PI / NTK / YaRN

Chen et al., 2023（PI）；bloc97, 2023（NTK-aware）；Peng et al., 2023（YaRN）

模型训练长度为 L，想要在推理时扩展到 sL，有三种主要方法：

- **PI**：把位置整体压缩为 p/s
- **NTK**：改大 base，高频分量几乎不变、低频分量被插值
- **YaRN**：按每个频率在原长度内转过的圈数 r_i 分段处理，高频外推、低频插值，中间平滑过渡，再对注意力温度做修正

$$\text{PI: } \theta_i' = \frac{\theta_i}{s} \qquad \text{NTK: } \text{base}' = \text{base} \cdot s^{\frac{d}{d-2}} \qquad \text{YaRN: } \theta_i' = (1-\gamma_i)\frac{\theta_i}{s} + \gamma_i\theta_i,\ \ \gamma_i = \text{clip}\Big(\frac{r_i-\alpha}{\beta-\alpha}, 0, 1\Big)$$

```
s, L0, base = 4.0, 4096, 10000.0                        # 扩展倍数、原训练长度
idx = torch.arange(0, d, 2).float() / d
inv = base ** (-idx)
inv_pi = inv / s                                        # PI：所有频率统一压缩
inv_ntk = (base * s ** (d / (d - 2))) ** (-idx)         # NTK-aware：改 base
r = L0 * inv / (2 * math.pi)                            # 每个频率在原长度内转过的圈数
ramp = ((r - 1) / (32 - 1)).clamp(0, 1)                 # YaRN：α=1, β=32
inv_yarn = inv_pi * (1 - ramp) + inv * ramp             # 高频外推、低频插值
mscale = 0.1 * math.log(s) + 1                          # 注意力温度修正（乘到 q、k 上）
print(inv[:3], inv_yarn[:3], mscale)
```

6.7 PaTH 注意力

Yang et al., 2025

RoPE 的旋转只依赖位置，与内容无关。PaTH 把旋转换成**数据依赖**的 Householder 变换的累积乘积：q_i 与 k_j 之间经过的每个 token 都会贡献一个变换。这样位置编码本身就能做状态追踪，表达能力更强，同时仍然可以高效并行。

$$s_{ij} = q_i^\top \left(\prod_{t=j+1}^{i} H_t \right) k_j, \qquad H_t = I - \beta_t w_t w_t^\top,\ \ \beta_t \in (0, 2)$$

```
w = F.normalize(torch.randn(B, H, N, d), dim=-1)          # 由 x_t 产生
beta = 2 * torch.sigmoid(torch.randn(B, H, N))
Hs = torch.eye(d) - beta[..., None, None] * w[..., :, None] * w[..., None, :]   # (B,H,N,d,d)
s = torch.full((B, H, N, N), float('-inf'))
for i in range(N):                                        # 朴素 O(N²d²) 实现，原文为分块并行算法
    P = torch.eye(d).expand(B, H, d, d)
    for j in range(i, -1, -1):
        s[:, :, i, j] = (q[:, :, i, None] @ P @ k[:, :, j, :, None]).flatten(2).squeeze(-1) / math.sqrt(d)
        P = P @ Hs[:, :, j]                               # 累积 H_i ... H_j
out = s.softmax(-1) @ v; print(out.shape)
```

第 7 章 记忆、检索与潜变量

让注意力看到当前窗口以外的信息有几种办法：缓存上一段的隐状态、压缩旧记忆、做 kNN 检索，或者引入一小组可学习的“潜变量”作为信息瓶颈。

7.1 段级循环（Transformer-XL）

Dai et al., 2019

把上一段的隐状态停止梯度后缓存下来，作为当前段额外的 K/V，从而突破固定的上下文长度。同时引入了相对位置打分的 4 项分解（内容–内容、内容–位置、全局内容偏置、全局位置偏置）。

$$\tilde{h}_{\tau}^{n-1} = \left[\mathrm{SG}(h_{\tau-1}^{n-1}) \,\|\, h_{\tau}^{n-1}\right], \qquad q = h_{\tau}^{n-1} W_Q,\ \ k, v = \tilde{h}_{\tau}^{n-1} W_{K,V}$$

```
Ml = 8
mem = torch.randn(B, H, Ml, d)                          # 上一段缓存的隐状态
kx, vx = torch.cat([mem.detach(), k], 2), torch.cat([mem.detach(), v], 2)
mask = torch.ones(N, Ml + N, dtype=torch.bool).tril(Ml)    # 记忆全可见 + 当前段因果
out = sdpa(q, kx, vx, mask); print(out.shape)
```

7.2 压缩记忆（Compressive Transformer）

Rae et al., 2020 · DeepMind

在 Transformer-XL 的基础上增加一级“压缩记忆”：从普通记忆中被挤出的旧激活，不直接丢弃，而是用池化或卷积按压缩率 c 压缩后保留下来。

$$\mathrm{cm} = f_c\left(\mathrm{mem}_{\mathrm{old}}\right) \in \mathbb{R}^{\lfloor n_s/c \rfloor \times d}, \qquad K, V \leftarrow [\mathrm{cm};\ \mathrm{mem};\ h]$$

```
c = 4
old = torch.randn(B, H, 8, d)                           # 即将被淘汰的旧记忆
cmem = F.avg_pool1d(old.flatten(0, 1).transpose(1, 2), c).transpose(1, 2).view(B, H, 8 // c, d)
mem = torch.randn(B, H, 8, d)
P = cmem.size(2) + mem.size(2)
kx, vx = torch.cat([cmem, mem, k], 2), torch.cat([cmem, mem, v], 2)
out = sdpa(q, kx, vx, torch.ones(N, P + N, dtype=torch.bool).tril(P)); print(out.shape)
```

7.3 kNN 记忆（Memorizing Transformer）

Wu et al., 2022 · ICLR

把历史上所有的 (k, v) 存进一个外部库（不参与反向传播），每个查询先做近似 kNN 检索出 top-k，再与局部注意力的结果通过一个门融合。

$$o = g \odot \mathrm{Attn}(q, K_{\mathrm{kNN}(q)}, V_{\mathrm{kNN}(q)}) + (1-g) \odot \mathrm{Attn}_{\mathrm{local}}(q, K, V)$$

```
Mk, Mv, tk = torch.randn(B, H, 1000, d), torch.randn(B, H, 1000, d), 8   # 外部 (k,v) 库
idx = (q @ Mk.transpose(-2, -1)).topk(tk, -1).indices      # 实际用 Faiss / ScaNN 近似检索
gat = lambda M: M.gather(2, idx.flatten(2)[..., None].expand(-1, -1, -1, d)).view(B, H, N, tk, d)
kk, vv = gat(Mk), gat(Mv)
o_mem = ((q[..., None, :] @ kk.transpose(-2, -1)) / math.sqrt(d)).softmax(-1) @ vv
o_loc = sdpa(q, k, v, causal)
g = torch.sigmoid(torch.zeros(H, 1, 1))                    # 每头可学习门
out = g * o_mem.squeeze(-2) + (1 - g) * o_loc; print(out.shape)
```

7.4 地标注意力（Landmark Attention）

Mohtashami & Jaggi, 2023

每个块插入一个“地标 token”，它的 key 代表整块的内容。注意力被分解为两级：先按地标的打分选块，再在块内做细粒度注意力。不需要的块可以留在 CPU 内存里，需要时再取回。

$$A_{ij} = \underbrace{\mathrm{softmax}_b(q_i^\top \ell_b)}_{\text{选块}} \cdot \underbrace{\mathrm{softmax}_{j\in b}(q_i^\top k_j)}_{\text{块内}}, \qquad b = \mathrm{block}(j)$$

```
bs = 4; nb = N // bs
kl = torch.randn(B, H, nb, d)                              # 每块的地标 key（训练得到）
p_blk = (q @ kl.transpose(-2, -1) / math.sqrt(d)).softmax(-1)          # (B,H,N,nb)
p_tok = (q @ k.transpose(-2, -1) / math.sqrt(d)).view(B, H, N, nb, bs).softmax(-1)
A = (p_blk[..., None] * p_tok).view(B, H, N, N)            # 块概率 × 块内概率
out = A @ v; print(out.shape)                              #  （示例为非因果简化版）
```

7.5 无限注意力（Infini-attention）

Munkhdalai et al., 2024 · Google

在每个注意力层里同时放两套机制：一个局部 softmax 注意力，一个用线性注意力维护的压缩记忆 M。每处理完一段就把这一段写入 M，两者的输出用可学习的门 β 混合。内存占用固定，可以流式处理无限长的输入。

$$A_{\mathrm{mem}} = \frac{\sigma(Q) M_{s-1}}{\sigma(Q) z_{s-1}}, \quad M_s = M_{s-1} + \sigma(K)^\top V, \quad A = \mathrm{sigmoid}(\beta)\, A_{\mathrm{mem}} + (1 - \mathrm{sigmoid}(\beta))\, A_{\mathrm{dot}}$$

```
phi = lambda t: F.elu(t) + 1
M, z, beta, outs = torch.zeros(B, H, d, d), torch.zeros(B, H, d, 1), torch.zeros(H, 1, 1), []
for seg in torch.randn(3, B, H, N, d):                    # 3 个段（这里令 q=k=v=seg 简化）
    Q = K = phi(seg)
    a_mem = (Q @ M) / (Q @ z + 1e-6)                      # 从压缩记忆读
    a_dot = sdpa(seg, seg, seg, causal)                   # 段内局部注意力
    outs.append(torch.sigmoid(beta) * a_mem + (1 - torch.sigmoid(beta)) * a_dot)
    M = M + K.transpose(-2, -1) @ seg                     # 写入记忆（原文还有 delta 版本）
    z = z + K.sum(2, keepdim=True).transpose(-2, -1)
out = torch.stack(outs); print(out.shape)
```

7.6 潜变量交叉注意力（Perceiver）

Jaegle et al., 2021 · DeepMind

用一组数量很少（$M \ll N$）的可学习潜向量作为查询，交叉注意力读入超长输入，复杂度为 $O(MN)$；之后只在潜空间里做自注意力。这种做法与输入的模态无关。Perceiver IO 和 Flamingo 的 Perceiver Resampler 都基于这个思路。

$$Z \leftarrow Z + \mathrm{Attn}(ZW_Q,\ XW_K,\ XW_V), \qquad Z \leftarrow Z + \mathrm{SelfAttn}(Z)$$

```
latents = nn.Parameter(torch.randn(8, D))                 # M=8 个潜向量
ca = nn.MultiheadAttention(D, H, batch_first=True)
sa = nn.MultiheadAttention(D, H, batch_first=True)
z = latents.expand(B, -1, -1)
z = z + ca(z, x, x)[0]                                    # 读：潜变量 ← 输入
z = z + sa(z, z, z)[0]                                    # 在小潜空间里自注意力
out = z; print(out.shape)
```

7.7 诱导点注意力（Set Transformer, ISAB）

Lee et al., 2019

处理集合数据时，先用 m 个可学习的“诱导点”汇总整个集合，再把汇总结果广播回每个元素，复杂度为 $O(nm)$，并且保持置换等变。

$$\mathrm{ISAB}(X) = \mathrm{MAB}(X,\ \mathrm{MAB}(I, X)), \qquad \mathrm{MAB}(A, B) = \mathrm{LN}(A + \mathrm{Attn}(A, B, B))$$

```
ind = nn.Parameter(torch.randn(1, 4, D))                  # 4 个诱导点
mab1 = nn.MultiheadAttention(D, H, batch_first=True)
mab2 = nn.MultiheadAttention(D, H, batch_first=True)
h = mab1(ind.expand(B, -1, -1), x, x)[0]                  # 诱导点汇总集合
out = mab2(x, h, h)[0]                                    # 广播回每个元素
print(out.shape)
```

7.8 槽注意力（Slot Attention）

Locatello et al., 2020 · Google

用于以对象为中心的表示学习。与普通注意力不同，softmax 在**槽（查询）维度**上做，各个槽互相竞争输入特征，每个槽最终“认领”一个对象。迭代若干轮，每轮用 GRU 更新槽。

$$A = \mathrm{softmax}_{\text{slots}}\left(\frac{(SW_Q)(XW_K)^\top}{\sqrt{D}}\right), \quad U = \frac{A}{\sum_j A_{:,j}}(XW_V), \quad S \leftarrow \mathrm{GRU}(S, U)$$

```
nS, iters = 4, 3
to_q, to_k, to_v = nn.Linear(D, D), nn.Linear(D, D), nn.Linear(D, D)
gru = nn.GRUCell(D, D)
slots, kx, vx = torch.randn(B, nS, D), to_k(x), to_v(x)
for _ in range(iters):
    A = (to_q(slots) @ kx.transpose(1, 2) / math.sqrt(D)).softmax(1)   # 在槽维 softmax：竞争
    U = (A / A.sum(-1, keepdim=True)) @ vx                             # 加权平均
    slots = gru(U.reshape(-1, D), slots.reshape(-1, D)).view(B, nS, D)
out = slots; print(out.shape)
```

第 8 章 视觉与图结构

在 Transformer 出现之前，CNN 里已经有通道注意力、空间注意力等“注意力”模块。视觉 Transformer 时代的主要问题是：图像 token 太多，全局自注意力太贵。图上的注意力则要处理不规则的邻接结构。

8.1 通道注意力（Squeeze-and-Excitation, SE）

Hu et al., 2018 · ILSVRC 2017 冠军

先全局池化把每个通道“挤压”成一个数，再用两层 MLP 加 sigmoid 生成通道权重，最后按通道重新缩放特征图。

$$s = \sigma\big(W_2\,\delta(W_1\,\mathrm{GAP}(X))\big), \qquad \tilde{X}_c = s_c \cdot X_c$$

```
img, C, r = torch.randn(B, 64, 8, 8), 64, 16
fc = nn.Sequential(nn.Linear(C, C // r), nn.ReLU(), nn.Linear(C // r, C), nn.Sigmoid())
out = img * fc(img.mean((2, 3)))[:, :, None, None]
print(out.shape)
```

8.2 空间注意力与 CBAM

Woo et al., 2018

CBAM 串联两个模块：

- **通道注意力**：同时使用平均池化和最大池化
- **空间注意力**：沿通道维取均值和最大值，拼接后用一个 7×7 卷积生成空间权重图

$$M_c = \sigma\big(\mathrm{MLP}(\mathrm{Avg}(X)) + \mathrm{MLP}(\mathrm{Max}(X))\big), \qquad M_s = \sigma\big(f^{7\times7}([\mathrm{Avg}_c(X'); \mathrm{Max}_c(X')])\big)$$

```
img, C, r = torch.randn(B, 64, 8, 8), 64, 16
mlp = nn.Sequential(nn.Linear(C, C // r), nn.ReLU(), nn.Linear(C // r, C))
ca = torch.sigmoid(mlp(img.mean((2, 3))) + mlp(img.amax((2, 3))))
y = img * ca[:, :, None, None]                          # 通道注意力
conv = nn.Conv2d(2, 1, 7, padding=3)
sa = torch.sigmoid(conv(torch.cat([y.mean(1, keepdim=True), y.amax(1, keepdim=True)], 1)))
out = y * sa; print(out.shape)                          # 空间注意力
```

8.3 非局部模块（Non-local Neural Networks）

Wang et al., 2018

把自注意力引入 CNN：每个像素对所有像素加权求和，用来捕捉长距离依赖。“嵌入高斯”版本就是标准的自注意力，以残差形式插入网络。

$$y_i = \frac{1}{\mathcal{C}(x)} \sum_j f(x_i, x_j)\, g(x_j), \qquad z_i = W_z y_i + x_i$$

```
img = torch.randn(B, 64, 8, 8)
th, ph, g, Wz = nn.Conv2d(64, 32, 1), nn.Conv2d(64, 32, 1), nn.Conv2d(64, 32, 1), nn.Conv2d(32, 64, 1)
flat = lambda t: t.flatten(2).transpose(1, 2)              # (B,HW,C')
y = (flat(th(img)) @ flat(ph(img)).transpose(1, 2)).softmax(-1) @ flat(g(img))
out = img + Wz(y.transpose(1, 2).view(B, 32, 8, 8)); print(out.shape)
```

8.4 图像块自注意力（ViT）

Dosovitskiy et al., 2021

把图像切成 $P \times P$ 的图像块（patch），每块线性投影成一个 token，加上位置编码和 [CLS] token，之后直接送入标准 Transformer。

$$z_0 = [x_{\text{cls}};\, x_p^1 E;\, \ldots;\, x_p^N E] + E_{\text{pos}}, \qquad z_\ell = z_{\ell-1} + \text{MHA}(\text{LN}(z_{\ell-1}))$$

```
img, P = torch.randn(B, 3, 32, 32), 8
patch = nn.Conv2d(3, D, P, stride=P)                       # 等价于切块 + 线性投影
tok = patch(img).flatten(2).transpose(1, 2)                # (B,16,D)
tok = torch.cat([torch.zeros(B, 1, D), tok], 1) + torch.randn(1, 17, D) * 0.02   # [CLS] + 位置编码
out = nn.TransformerEncoderLayer(D, H, batch_first=True)(tok); print(out.shape)
```

8.5 轴向注意力（Axial Attention）

Ho et al., 2019；Wang et al., 2020（Axial-DeepLab）

把二维注意力拆成“先沿行、再沿列”两次一维注意力，复杂度从 $O((HW)^2)$ 降到 $O(HW(H+W))$，两次之后仍然能覆盖全图。

$$Y = \text{Attn}_{\text{col}}(\text{Attn}_{\text{row}}(X))$$

```
Hh = Ww = 8
feat = torch.randn(B, Hh, Ww, D)
row_attn = nn.MultiheadAttention(D, H, batch_first=True)
col_attn = nn.MultiheadAttention(D, H, batch_first=True)
r = feat.reshape(B * Hh, Ww, D); r = row_attn(r, r, r)[0].view(B, Hh, Ww, D)          # 沿宽（行内）
c = r.transpose(1, 2).reshape(B * Ww, Hh, D); c = col_attn(c, c, c)[0]                 # 沿高（列内）
out = c.view(B, Ww, Hh, D).transpose(1, 2); print(out.shape)
```

8.6 十字交叉注意力（Criss-Cross Attention, CCNet）

Huang et al., 2019

每个像素只关注与自己同行或同列的像素，形成十字形的注意力范围。循环两次之后，任意两个像素之间都能传递信息。

$$\mathcal{A}_{(u,v)} = \{(u, v') : \forall v'\} \cup \{(u', v) : \forall u'\}, \qquad Y = \mathrm{CC}(\mathrm{CC}(X))$$

```
Hh = Ww = 4; Pn = Hh * Ww
ri, ci = torch.arange(Pn) // Ww, torch.arange(Pn) % Ww
cc = (ri[:, None] == ri[None]) | (ci[:, None] == ci[None]) # 十字形掩码
t = torch.randn(B, H, Pn, d)
y = sdpa(t, t, t, cc)
out = sdpa(y, y, y, cc); print(out.shape)                  # 循环两次 → 全图感受野
```

8.7 窗口与移位窗口注意力（Swin Transformer）

Liu et al., 2021 · ICCV 最佳论文

先把特征图切成不重叠的 $w \times w$ 窗口，在每个窗口内部做注意力（W-MSA），复杂度与图像大小成线性。下一层把窗口整体移位 $w/2$（SW-MSA），让相邻窗口之间能交换信息。

$$\text{W-MSA: Attn within each } w \times w \text{ window;} \qquad \text{SW-MSA: } \mathrm{roll}\big(X, -\tfrac{w}{2}\big) \to \text{W-MSA} \to \mathrm{roll}\big(\cdot, +\tfrac{w}{2}\big)$$

```
Hh = Ww = 8; ws = 4
feat = torch.randn(B, Hh, Ww, D)
mha = nn.MultiheadAttention(D, H, batch_first=True)
def win_attn(f):
    w = f.view(B, Hh // ws, ws, Ww // ws, ws, D).permute(0, 1, 3, 2, 4, 5).reshape(-1, ws * ws, D)
    w = mha(w, w, w)[0]
    return w.view(B, Hh // ws, Ww // ws, ws, ws, D).permute(0, 1, 3, 2, 4, 5).reshape(B, Hh, Ww, D)
y = win_attn(feat)                                         # W-MSA
out = torch.roll(win_attn(torch.roll(y, (-ws // 2, -ws // 2), (1, 2))), (ws // 2, ws // 2), (1, 2))
print(out.shape)                                           # SW-MSA（省略了跨边界的掩码）
```

8.8 可变形注意力（Deformable Attention）

Zhu et al., 2021 · Deformable DETR

每个查询只在参考点附近采样 K 个点（通常 $K = 4$），采样偏移和注意力权重都由查询直接预测，**不计算 QK 点积**。复杂度与特征图大小无关，收敛也比 DETR 快 10 倍。

$$\mathrm{DeformAttn}(z_q, p_q, X) = \sum_{h=1}^{H} W_h \left[\sum_{k=1}^{K} A_{hqk} \cdot W'_h X(p_q + \Delta p_{hqk}) \right]$$

```
feat, Nq, Kp = torch.randn(B, D, 8, 8), 10, 4              # 特征图、查询数、每头采样点数
query, ref = torch.randn(B, Nq, D), torch.rand(B, Nq, 2)   # 查询与参考点（归一化坐标）
off = nn.Linear(D, H * Kp * 2)(query).view(B, Nq, H, Kp, 2) * 0.1
A = nn.Linear(D, H * Kp)(query).view(B, Nq, H, Kp).softmax(-1)          # 权重直接由查询预测
grid = ((ref[:, :, None, None] + off) * 2 - 1).permute(0, 2, 1, 3, 4).reshape(B * H, Nq, Kp, 2)
val = nn.Conv2d(D, D, 1)(feat).view(B * H, d, 8, 8)        # 每头的值特征
samp = F.grid_sample(val, grid, align_corners=False)       # 双线性采样（B·H,d,Nq,Kp）
out = (samp * A.permute(0, 2, 1, 3).reshape(B * H, 1, Nq, Kp)).sum(-1)
out = out.view(B, H * d, Nq).transpose(1, 2); print(out.shape)
```

8.9 图注意力网络（GAT / GATv2）

Veličković et al., 2018；Brody et al., 2022（GATv2）

每个节点只对自己的邻居做注意力，打分用“拼接 + LeakyReLU”。GATv2 把非线性挪到 $a^\top$ 之前，使注意力变为“动态”的，表达能力更强。

$$
\begin{aligned}
\text{GAT:}\quad & e_{ij} = \mathrm{LeakyReLU}\big(a^\top [W h_i \,\|\, W h_j]\big) \\
\text{GATv2:}\quad & e_{ij} = a^\top \mathrm{LeakyReLU}\big(W [h_i \,\|\, h_j]\big) \\
& \alpha_{ij} = \mathrm{softmax}_{j \in \mathcal{N}(i)}(e_{ij})
\end{aligned}
$$

```
Nn, Fin, Fo = 6, 16, 8
h = torch.randn(Nn, Fin)
adj = (torch.rand(Nn, Nn) < 0.4) | torch.eye(Nn, dtype=torch.bool)   # 邻接矩阵（含自环）
W, a = nn.Linear(Fin, Fo, bias=False), torch.randn(2 * Fo)
Wh = W(h)
e = F.leaky_relu((Wh @ a[:Fo])[:, None] + (Wh @ a[Fo:])[None], 0.2)
alpha = e.masked_fill(~adj, float('-inf')).softmax(-1)     # 只在邻居上归一化
out = alpha @ Wh; print(out.shape)
```

8.10 图 Transformer（Graphormer）

Ying et al., 2021 · OGB-LSC 冠军

对图上所有节点做全局注意力，把图结构编码成注意力偏置：最短路径距离 $\phi(v_i, v_j)$ 映射成每头一个可学习标量，再加上中心性编码（按节点度数）和边编码。

$$
A_{ij} = \frac{(h_i W_Q)(h_j W_K)^\top}{\sqrt{d}} + b_{\phi(v_i, v_j)} + c_{ij}
$$

```
Nn = 6
Adj = (torch.rand(Nn, Nn) < 0.3).float(); Adj = ((Adj + Adj.T + torch.eye(Nn)) > 0).float()
spd, reach = torch.full((Nn, Nn), 99), torch.eye(Nn)       # 99 = 不可达
for hop in range(Nn):                                      # BFS 求最短路径距离
    spd[(reach > 0) & (spd == 99)] = hop
    reach = ((reach @ Adj) > 0).float()
bias = nn.Embedding(100, H)(spd).permute(2, 0, 1)          # 距离 → 每头偏置（H,Nn,Nn）
nodes = torch.randn(1, H, Nn, d)
out = sdpa(nodes, nodes, nodes, bias=bias); print(out.shape)
```

8.11 扩散模型中的交叉注意力（Latent Diffusion）

Rombach et al., 2022 · Stable Diffusion 1.x/2.x

U-Net 中的图像 latent token 作为查询，文本编码器（如 CLIP）的输出作为 K/V，用这种方式把文本条件注入图像生成。Prompt-to-Prompt 等图像编辑方法就是直接操作这张交叉注意力图。

$$\mathrm{Attn}\big(\varphi(z_t)W_Q,\ \tau_\theta(y)W_K,\ \tau_\theta(y)W_V\big)$$

```
img_tok, text = torch.randn(B, 64, D), torch.randn(B, 77, 768)   # 64 个 latent token、77 个文本 token
Wq, Wk, Wv = nn.Linear(D, D), nn.Linear(768, D), nn.Linear(768, D)
out = merge(sdpa(split(Wq(img_tok)), split(Wk(text)), split(Wv(text)))); print(out.shape)
```

8.12 多模态联合注意力（MM-DiT）

Esser et al., 2024 · Stable Diffusion 3；FLUX 使用

图像和文本两种模态各自使用独立的投影权重，但把序列拼在一起，做**一次**联合自注意力。信息双向流动：文本 token 也会根据图像不断更新。

$$[Q;K;V] = \big[\,X_{\text{img}}W^{\text{img}}_{QKV}\,;\,X_{\text{txt}}W^{\text{txt}}_{QKV}\,\big],\qquad O = \mathrm{Attn}(Q,K,V)$$

```
img, txt = torch.randn(B, 64, D), torch.randn(B, 20, D)
qkv_i, qkv_t = nn.Linear(D, 3 * D), nn.Linear(D, 3 * D)    # 两种模态各自的投影
qi, ki, vi = qkv_i(img).chunk(3, -1); qt, kt, vt = qkv_t(txt).chunk(3, -1)
cat = lambda a, b: split(torch.cat([a, b], 1))
o = merge(sdpa(cat(qi, qt), cat(ki, kt), cat(vi, vt)))     # 拼接后做一次联合注意力
img_out, txt_out = o[:, :64], o[:, 64:]
out = o; print(img_out.shape, txt_out.shape)
```

第 9 章 系统实现层

本章的方法在**数学上与标准注意力完全等价**，区别只在于如何在硬件上计算：减少显存读写，切分到多个设备上，或者降低数值精度。

9.1 FlashAttention（v1 → v4）

Dao et al., 2022（v1）、2023（v2）；Shah et al., 2024（v3, Hopper）；2025（v4, Blackwell）

主要做法：

- **分块（tiling）加在线 softmax**：$N \times N$ 的注意力矩阵从不写回显存（HBM），只在片上 SRAM 里流式计算，显存占用从 $O(N^2)$ 降到 $O(N)$
- **反向传播时重算**注意力矩阵，而不是保存它
- 各版本的改进：v2 优化了并行与工作划分；v3 利用 Hopper 的异步执行和 FP8；v4 面向 Blackwell 架构

$$m' = \max(m,\ \mathrm{rowmax}(S_j)), \qquad \ell' = e^{m-m'}\ell + \mathrm{rowsum}(e^{S_j - m'}),$$
$$O' = e^{m-m'}O + e^{S_j-m'}V_j, \qquad O_{\text{final}} = O/\ell$$

```
def flash_attn(q, k, v, bk=4):
    m = torch.full(q.shape[:-1] + (1,), float('-inf'))     # 运行中的行最大值
    l, o = torch.zeros_like(m), torch.zeros_like(q)        # 运行中的分母、未归一化输出
    for s0 in range(0, k.size(-2), bk):                    # 按 K/V 块流式处理
        s = q @ k[..., s0:s0 + bk, :].transpose(-2, -1) / math.sqrt(q.size(-1))
        m_new = torch.maximum(m, s.amax(-1, keepdim=True))
        p, c = torch.exp(s - m_new), torch.exp(m - m_new)  # c：旧结果的重缩放因子
        l, o, m = c * l + p.sum(-1, keepdim=True), c * o + p @ v[..., s0:s0 + bk, :], m_new
    return o / l
out = flash_attn(q, k, v)
print(torch.allclose(out, sdpa(q, k, v), atol=1e-5))
```

9.2 分页注意力（PagedAttention）

Kwon et al., 2023 · vLLM

借鉴操作系统的虚拟内存分页：把 KV cache 存在固定大小的物理块中，每个序列用一张“块表”把逻辑块映射到物理块。这样几乎没有显存碎片，不同请求还可以共享前缀（写时复制），推理吞吐提升 2–4 倍。

$$K_{\text{seq}} = \left[\, \mathcal{P}_K[\text{bt}[0]];\ \mathcal{P}_K[\text{bt}[1]];\ \dots \,\right]_{:L}, \qquad o = \text{Attn}(q_t, K_{\text{seq}}, V_{\text{seq}})$$

```
blk, nphys = 4, 32
k_pool, v_pool = torch.randn(nphys, H, blk, d), torch.randn(nphys, H, blk, d)   # 物理块池
block_table = torch.tensor([7, 2, 19, 5])                 # 某序列的逻辑块 → 物理块（不必连续）
L = 14                                                    # 序列实际长度
kc = k_pool[block_table].permute(1, 0, 2, 3).reshape(H, -1, d)[:, :L]
vc = v_pool[block_table].permute(1, 0, 2, 3).reshape(H, -1, d)[:, :L]
out = sdpa(torch.randn(H, 1, d), kc, vc); print(out.shape)  # 解码一步：单个查询
```

9.3 环形注意力（Ring Attention）

Liu et al., 2023

用于序列/上下文并行：每个设备持有一段 Q，各段 K/V 在设备之间沿环形依次传递；每收到一块就用在线 softmax 累加。通信可以被计算完全覆盖，上下文长度随设备数线性扩展。

$$O_r = \text{OnlineSoftmax}\Big(\big\{\, Q_r K_{(r-s) \bmod P}^{\top} \,\big\}_{s=0}^{P-1}\Big)$$

```
P = 4                                                     # 设备数
qs, ks, vs = q.chunk(P, 2), k.chunk(P, 2), v.chunk(P, 2)
outs = []
for r in range(P):                                        # 设备 r 持有 qs[r]
    m = torch.full(qs[r].shape[:-1] + (1,), float('-inf')); l = torch.zeros_like(m); o = torch.zeros_like(qs[r])
    for step in range(P):                                 # 每一步从邻居收到下一块 K/V
        j = (r - step) % P
        s = qs[r] @ ks[j].transpose(-2, -1) / math.sqrt(d)
        m_new = torch.maximum(m, s.amax(-1, keepdim=True)); p = torch.exp(s - m_new); c = torch.exp(m - m_new)
        l, o, m = c * l + p.sum(-1, keepdim=True), c * o + p @ vs[j], m_new
    outs.append(o / l)
out = torch.cat(outs, 2)
print(torch.allclose(out, sdpa(q, k, v), atol=1e-5))
```

9.4 条纹注意力（Striped Attention）

Brandon et al., 2023

在因果掩码下，Ring Attention 按连续段切分会造成负载不均：越靠后的设备算得越多。条纹注意力改为把 token 轮流分配给各设备（第 i 个 token 分给设备 $i \bmod P$），每个设备分到的因果三角形面积几乎相同。

$$\mathcal{T}_r = \{\, i : i \bmod P = r \,\}$$

```
P = 4
contig = [torch.arange(N)[r * N // P:(r + 1) * N // P] for r in range(P)]   # Ring: 连续切分
stripe = [torch.arange(N)[r::P] for r in range(P)]                          # Striped: 轮转切分
work = lambda part: [int(causal[idx].sum()) for idx in part]                # 每个设备要算的 (i,j) 对数
print("连续:", work(contig), " 条纹:", work(stripe))
```

9.5 DeepSpeed-Ulysses

Jacobs et al., 2023 · 微软

另一种序列并行方案：输入按序列切分；在注意力之前做一次 all-to-all 通信，变成“每个设备持有完整序列、H/P 个头”；各设备独立计算完整的注意力，再做一次 all-to-all 换回按序列切分。通信量与设备数无关，但并行度受头数限制。

$$[N/P,\ H] \xrightarrow{\text{all-to-all}} [N,\ H/P] \xrightarrow{\text{Attn}} [N,\ H/P] \xrightarrow{\text{all-to-all}} [N/P,\ H]$$

```
P = 2
def a2a(t):                                          # 模拟 all-to-all：按序列切 → 按头切
    shards = t.chunk(P, 2)
    return [torch.cat([s[:, h:h + H // P] for s in shards], 2) for h in range(0, H, H // P)]
outs = [sdpa(qh, kh, vh) for qh, kh, vh in zip(a2a(q), a2a(k), a2a(v))]   # 每设备算完整序列
out = torch.cat(outs, 1)                             # （再 all-to-all 回按序列切分）
print(torch.allclose(out, sdpa(q, k, v), atol=1e-5))
```

9.6 FlexAttention

PyTorch 团队, 2024 · PyTorch ⩾2.5

用户只需写两个小函数：`score_mod`（修改单个打分）和 `mask_mod`（决定是否可见）。`torch.compile` 会把它们融合进一个 FlashAttention 风格的 kernel，并根据掩码自动跳过空块。ALiBi、滑窗、Prefix-LM、文档掩码、软截断（softcapping）等变体都可以用几行代码实现。

$$O = \text{softmax}\big(\text{score_mod}(S_{ij}, b, h, i, j) \text{ if } \text{mask_mod}(b, h, i, j) \text{ else } -\infty\big)V$$

```
from torch.nn.attention.flex_attention import flex_attention, create_block_mask
def alibi(score, b, h, q_idx, kv_idx):                   # score_mod：对每个打分做变换
    return score - (q_idx - kv_idx) * 2.0 ** (-(h + 1))
def causal_mask(b, h, q_idx, kv_idx):                    # mask_mod：是否可见
    return q_idx >= kv_idx
bm = create_block_mask(causal_mask, None, None, N, N, device='cpu')
out = flex_attention(q, k, v, score_mod=alibi, block_mask=bm)   # GPU 上配合 torch.compile 生成融合 kernel
m_ = 2.0 ** (-(torch.arange(H) + 1.0))
print(torch.allclose(out, sdpa(q, k, v, causal, -m_[:, None, None] * (I - J)), atol=1e-4))
```

9.7 内存高效注意力（Memory-Efficient Attention / SDPA 后端）

Rabe & Staats, 2021；xFormers；PyTorch SDPA

Rabe 和 Staats 证明了自注意力的显存可以做到 $O(\sqrt{N})$ 甚至 $O(\log N)$，xFormers 据此实现了内存高效注意力。今天只需调用 `F.scaled_dot_product_attention`，PyTorch 会在 Flash、Efficient、cuDNN、Math 几种后端之间自动选择，也可以手动指定。

$$O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V \quad \text{（同一公式，不同 kernel）}$$

```
from torch.nn.attention import sdpa_kernel, SDPBackend
with sdpa_kernel([SDPBackend.MATH]):                    # GPU 上可选 FLASH_ATTENTION / EFFICIENT_ATTENTION /
↪ CUDNN_ATTENTION
    out = F.scaled_dot_product_attention(q, k, v, is_causal=True)
print(torch.allclose(out, sdpa(q, k, v, causal), atol=1e-5))
```

9.8 量化注意力（SageAttention）

Zhang et al., 2025 · 清华 · ICLR

把 Q、K 量化成 INT8 来计算 $QK^\top$，PV 部分用 FP16 或 FP8。关键技巧是对 K 做**平滑**：减去所有 token 的均值。这一步不改变 softmax 的结果（每行加同一个常数），但去除了离群值，降低了量化误差。速度比 FlashAttention2 快 2 倍以上，几乎没有精度损失。

$$S = \frac{\psi^{-1}\left(\psi_Q(Q)\,\psi_K(K-\bar{K})^\top\right)}{\sqrt{d}}, \qquad \mathrm{softmax}(Q(K-\bar{K})^\top) = \mathrm{softmax}(QK^\top)$$

```
def int8(t):
    sc = t.abs().amax(-1, keepdim=True) / 127           # 逐 token 缩放（原文为逐块）
    return (t / sc).round().clamp(-127, 127), sc
ks = k - k.mean(2, keepdim=True)                        # K 平滑：去离群值，softmax 不变
qi, sq = int8(q); ki, sk = int8(ks)
s = (qi @ ki.transpose(-2, -1)) * sq * sk.transpose(-2, -1) / math.sqrt(d)   # INT8 矩阵乘后反量化
out = s.softmax(-1) @ v                                 # PV 用 FP16/FP8
print("最大误差:", (out - sdpa(q, k, v)).abs().max().item())
```

第 10 章 混合架构

2025 年以后，前沿模型的趋势是**不再让每一层都用同一种注意力**。大部分层用便宜的机制（线性注意力、SSM 或滑窗），少数层保留全注意力，负责精确的检索与回忆。

10.1 线性注意力与全注意力交错

Jamba（AI21, 2024：Mamba : 注意力 ≈ 7:1）；MiniMax-01（2025：Lightning : softmax = 7:1）；Qwen3-Next（2025：Gated DeltaNet : 门控注意力 = 3:1）；Kimi Linear（2025：KDA : MLA = 3:1）

线性层或 SSM 层负责高效处理长序列，状态大小固定；每隔几层插入一个全注意力层，弥补线性注意力在“大海捞针”式精确检索上的短板。整体 KV cache 大约降为原来的 $1/(r+1)$。

$$\text{Layer}_\ell = \begin{cases} \text{Softmax Attn} & \ell \bmod (r+1) = r \\ \text{Linear Attn/SSM} & \text{否则} \end{cases}$$

```
def linear_attn(q, k, v):                                  # 可以换成第 4 章的任一方法
    Q, K = F.elu(q) + 1, F.elu(k) + 1
    A = (Q @ K.transpose(-2, -1)) * causal
    return (A @ v) / A.sum(-1, keepdim=True)
pattern = ['linear', 'linear', 'linear', 'full'] * 2       # 3:1 交错
h = x
for kind in pattern:
    qh, kh, vh = split(h), split(h), split(h)              # 简化：省略各层投影与 FFN
    y = linear_attn(qh, kh, vh) if kind == 'linear' else sdpa(qh, kh, vh, causal)
    h = h + merge(y)
out = h; print(out.shape)
```

10.2 局部与全局注意力交错

Gemma 2（2024，滑窗 : 全局 = 1:1）；Gemma 3（2025，5:1）；gpt-oss（2025，交替使用带 sink 的滑窗层和全注意力层）

用滑窗注意力（SWA）层承担大部分计算，KV cache 被窗口大小限住；少量全局层保证长距离信息能够传递。

$$\text{Layer}_\ell = \begin{cases} \text{Global Attn} & \ell \bmod (r+1) = r \\ \text{SWA}(w) & \text{否则} \end{cases}$$

```
w = 4
masks = {'local': causal & (I - J < w), 'global': causal}
pattern = ['local'] * 5 + ['global']                    # Gemma 3 风格 5:1
h = x
for kind in pattern:
    h = h + merge(sdpa(split(h), split(h), split(h), masks[kind]))
out = h; print(out.shape)
```

附录 速查表

类别	机制	训练复杂度	推理 KV / 状态	代表模型 / 用途
基础	MHA	$O(N^2d)$	$O(NHd)$	原始 Transformer、BERT
KV 压缩	MQA / GQA	$O(N^2d)$	$O(NGd)$	PaLM / Llama、Qwen、Mistral
KV 压缩	MLA	$O(N^2d)$	$O(N(d_c+d_r))$	DeepSeek-V2/V3/R1、Kimi K2
KV 压缩	TPA	$O(N^2d)$	$O(NR(H+d))$	T6
稀疏（固定）	SWA	$O(Nwd)$	$O(wHd)$	Mistral、Gemma 2/3
稀疏（固定）	Longformer / BigBird	$O(Nwd)$	—	长文档编码
稀疏（动态）	NSA	$\approx O(N(N/b+nb+w)d)$	$O(NHd)$	DeepSeek（研究）
稀疏（动态）	MoBA	$O(Nkbd)$	$O(NHd)$	Kimi 长上下文
稀疏（动态）	DSA	$O(N^2d_I)+O(Nkd)$	$O(N(d_c+d_r))$	DeepSeek-V3.2
线性	Linear / Performer	$O(Nd^2)$	$O(d^2)$	早期高效 Transformer
线性	RetNet / GLA	$O(Nd^2)$	$O(d^2)$	RetNet、GLA
线性	Gated DeltaNet / KDA	$O(Nd^2)$	$O(d^2)$	Qwen3-Next、Kimi Linear
线性	Lightning	$O(Nd^2)$	$O(d^2)$	MiniMax-01 / M1
SSM	Mamba-2 / SSD	$O(Nd\,d_s)$	$O(d\,d_s)$	Jamba、Nemotron-H 等混合模型
亚二次	Log-Linear	$O(Nd^2\log N)$	$O(d^2\log N)$	研究
测试时学习	TTT / Titans	$O(Nd^2)$	$O(\|W\|)$	研究
改造	Gated Attention	$O(N^2d)$	$O(NHd)$	Qwen3-Next

类别	机制	训练复杂度	推理 KV / 状态	代表模型 / 用途
改造	DIFF / FoX / Stick-breaking	$O(N^2d)$	$O(NHd)$	研究
位置	RoPE (+YaRN)	—	—	几乎所有 LLM
位置	ALiBi	—	—	BLOOM、MPT
系统	FlashAttention	$O(N^2d)$，显存 $O(N)$	—	所有训练与推理框架
系统	PagedAttention	—	分页、无碎片	vLLM、SGLang
系统	Ring / Ulysses	跨设备切分 N	—	百万 token 训练

一句话总结演进脉络

1. **2014–2017**：从 RNN 上的加性注意力，发展到 Transformer 的多头缩放点积注意力。
2. **2019–2021**：二次复杂度是瓶颈，于是出现固定稀疏模式、核化线性注意力、低秩近似。
3. **2022–2023**：FlashAttention 让精确注意力本身变快；MQA/GQA 解决推理时的 KV cache 问题。
4. **2024**：线性注意力加门控并获得硬件友好的分块算法（GLA、Mamba-2），MLA 压缩 KV。
5. **2025–2026**：可训练的动态稀疏（NSA、MoBA、DSA）、delta 规则类 RNN（Gated DeltaNet、KDA）、输出门控（Gated Attention），以及“少量全注意力 + 大量线性层”的混合架构成为前沿主流。